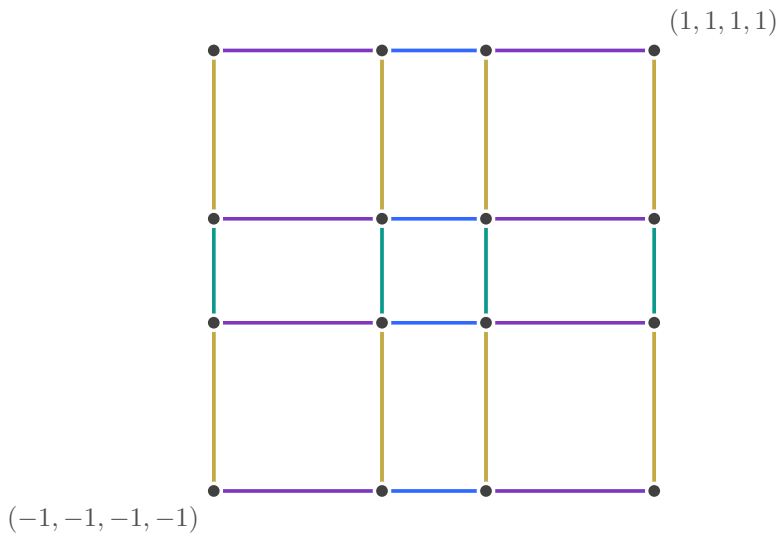
Boolean Fourier Analysis and the Walsh–Hadamard Transform

An Expository Guide



The Boolean hypercube $\{-1, +1\}^4$: 16 vertices, 32 edges, four dimensions

From First Principles to Spectral Methods for Binary Neural Networks

Companion to the thesis:
Spectral Methods for Binary Neural Networks

March 2026

Dedicated to the 2^n *inhabitants of* $\{-1, +1\}^n$.

"Mathematics compares the most diverse phenomena and discovers the secret analogies that unite them."

— Joseph Fourier, *Théorie analytique de la chaleur* (1822)

"In mathematics you don't understand things. You just get used to them."

— John von Neumann

"The purpose of computing is insight, not numbers."

— Richard Hamming, *Numerical Methods for Scientists and Engineers* (1962)

Contents

Roadmap **6**

1 Prerequisites **8**

1.1 Inner Product Spaces 8

1.2 The Kronecker Product 9

1.3 Probability Essentials 10

1.4 The Central Limit Theorem and Berry–Esséen 10

1.5 L^p Norms 11

2 Fourier Analysis on the Boolean Hypercube **13**

2.1 The Boolean Hypercube 13

2.2 The Parity Functions 14

2.3 Orthonormality 15

2.4 The Fourier Expansion 16

2.5 Parseval's Identity 16

2.6 Worked Examples 17

2.6.1 Dictator Functions 17

2.6.2 The AND Function 17

2.6.3 The Majority Function 18

2.6.4 The Parity Function (Anti-Concentration) 18

2.7 Degree-Level Energy Decomposition 19

2.8 Low-Degree Truncation 19

2.9 Plancherel and the Convolution Theorem 20

3 Influence and Noise Sensitivity **22**

3.1 Influence of Variables 22

3.2 The Noise Operator 23

3.3 Noise Stability 25

3.4 Relationship Between Influence and Noise Sensitivity 26

4 Hypercontractivity and Spectral Concentration **28**
4.1 The Bonami–Beckner Hypercontractivity Inequality 28
4.2 The Level-k Inequality 29
4.3 The LMN Theorem 31
4.3.1 Background: Boolean Circuits 31
4.3.2 The Theorem 32
4.4 Learning Fourier-Sparse Functions 33

5 The Walsh–Hadamard Transform **35**
5.1 Hadamard Matrices 35
5.2 Properties of Hadamard Matrices 35
5.3 The Walsh–Hadamard Transform 37
5.4 The Fast Walsh–Hadamard Transform 38
5.4.1 The Butterfly Algorithm 38
5.5 Comparison with the FFT 40
5.6 The Convolution Theorem via the WHT 40
5.7 The WHT as a Neural Network Layer 40

6 Linear Threshold Functions, Codes, and Groups **42**
6.1 Linear Threshold Functions 42
6.1.1 Chow's Theorem 43
6.1.2 Spectral Properties of LTFs 43
6.2 Reed–Muller Codes 44
6.2.1 Definition 45
6.2.2 Connection to the Thesis 46
6.3 The Group-Theoretic Perspective 47
6.3.1 The Group $\mathbb{Z}_2^n$ 47
6.3.2 Characters of Finite Abelian Groups 47
6.3.3 Comparison Table 48

7 Application to Binary Neural Networks **50**
7.1 Background: Binary Neural Networks 50
7.1.1 Standard Neural Networks 50
7.1.2 The Transformer Architecture 50
7.1.3 Binary Weight Networks 50
7.1.4 The Straight-Through Estimator 51
7.1.5 BitNet 51
7.2 The Spectral Perspective on Binary Weights 52
7.2.1 The Row-Wise Encoding 52
7.2.2 Spectral Parameterization 52

7.2.3 Normalization Without Learned Parameters 53
7.3 Tracing the Expository Chain 53
7.3.1 Chapter-by-Chapter Map 53
7.4 Worked Example: Spectral Parameterization of One Row 55
7.5 Concept Dependency Graph 56
7.6 Suggestions for Further Reading 56

Index **63**

Roadmap

This guide develops the mathematical background for the thesis *Spectral Methods for Binary Neural Networks* in seven chapters, each building on the previous ones.

Chapter 1: Prerequisites.

Inner product spaces, the Kronecker product, and the probability tools (Markov/Chebyshev inequalities, CLT, Berry–Esséen) used later. Skip this if you are comfortable with undergraduate linear algebra and probability.

Chapter 2: Fourier Analysis on the Boolean Hypercube.

The core of the guide. We define the parity basis, prove orthonormality, derive the Fourier expansion and Parseval's identity, and work through detailed examples (dictator, AND, majority, parity). We also prove the sign-rounding error bound used in the thesis.

Chapter 3: Influence and Noise Sensitivity.

Influence of variables, total influence, the noise operator T_ρ, noise stability, and their spectral formulas. These quantify the sensitivity of Boolean functions to perturbations.

Chapter 4: Hypercontractivity and Spectral Concentration.

The Bonami–Beckner inequality, the level-k inequality, and the LMN theorem. These are the main tools for proving that "simple" Boolean functions concentrate their spectral energy on low degrees.

Chapter 5: The Walsh–Hadamard Transform.

Hadamard matrices, the WHT as the computational realization of the Fourier expansion, the fast algorithm (FWHT), and its comparison with the FFT.

Chapter 6: Linear Threshold Functions, Codes, and Groups.

Three topics connecting the theory to the thesis: LTFs and Chow's theorem, Reed–Muller codes and spectral truncation, and the group-theoretic perspective on the Walsh–Fourier basis.

Chapter 7: Application to Binary Neural Networks.

How all the preceding material applies to the thesis: the spectral parameterization, normalization, and a chapter-by-chapter map from this guide to the thesis.

Dependencies. Chapters 1–2 are foundational and should be read first. Chapter 3 depends on 2. Chapter 4 depends on 3. Chapter 5 depends on 1–2 but is largely independent of 3–4. Chapter 6 depends on 2 and 5. Chapter 7 draws on all preceding chapters. See also the dependency graph in Section 7.5.

Chapter 1

Prerequisites

We collect the facts from linear algebra and probability that are used throughout this guide. The reader who is comfortable with inner product spaces, the Kronecker product, Markov's inequality, and the central limit theorem may safely skip to Chapter 2.

Standard references: Axler [2024] or Horn and Johnson [2012] for linear algebra; Durrett [2019] or Feller [1971] for probability.

1.1 Inner Product Spaces

Definition 1.1.1 (Inner product)**.** Let V be a real vector space. An *inner product* on V is a function $\langle \cdot, \cdot \rangle \colon V \times V \to \mathbb{R}$ satisfying, for all $u, v, w \in V$ and $\alpha \in \mathbb{R}$:

(i) *Symmetry:* $\langle u, v \rangle = \langle v, u \rangle$.

(ii) *Linearity in the first argument:* $\langle \alpha u + v, w \rangle = \alpha \langle u, w \rangle + \langle v, w \rangle$.

(iii) *Positive definiteness:* $\langle v, v \rangle \geq 0$, with equality if and only if $v = 0$.

The pair $(V, \langle \cdot, \cdot \rangle)$ is called an *inner product space*.

The inner product induces a norm $\|v\| = \sqrt{\langle v, v \rangle}$. The key structural fact is:

Theorem 1.1.2 (Cauchy–Schwarz inequality)**.** *For all u, v in an inner product space, $|\langle u, v \rangle| \leq \|u\| \, \|v\|$, with equality if and only if u and v are linearly dependent.*[1]

Definition 1.1.3 (Orthonormal basis)**.** A set $\{e_1, \ldots, e_n\}$ in an inner product space V is *orthonormal* if $\langle e_i, e_j \rangle = \delta_{ij}$ (the Kronecker delta). If additionally $\operatorname{span}\{e_1, \ldots, e_n\} = V$, it is an *orthonormal basis* (ONB).

The central fact about orthonormal bases is:

[1] Proof: see Axler [2024, Chapter 6] or Horn and Johnson [2012, Section 5.1].

Proposition 1.1.4 (Fourier expansion in a finite-dimensional inner product space)**.** *If* $\{e_1, \ldots, e_N\}$ *is an ONB for* $(V, \langle \cdot, \cdot \rangle)$*, then every* $v \in V$ *has a unique expansion*

$$v = \sum_{i=1}^{N} \langle v, e_i \rangle \, e_i,$$

and Parseval's identity *holds:* $\|v\|^2 = \sum_{i=1}^{N} \langle v, e_i \rangle^2$.

Proof. Write $v = \sum_i c_i e_i$. Taking the inner product with e_j gives $\langle v, e_j \rangle = c_j$ by orthonormality. Then $\|v\|^2 = \langle v, v \rangle = \sum_i c_i^2$. □

This is exactly the abstract template for the Boolean Fourier expansion in Chapter 2: the vector space will be the space of all real-valued functions on $\{-1, +1\}^n$, the inner product will be the expectation inner product, and the ONB will be the parity functions.

1.2 The Kronecker Product

Definition 1.2.1 (Kronecker product)**.** Let A be an $m \times n$ matrix and B a $p \times q$ matrix. Their *Kronecker product* $A \otimes B$ is the $mp \times nq$ block matrix

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix}.$$

Key properties (all straightforward to verify; see Horn and Johnson, 2012, Chapter 4 for proofs):

(i) $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$, when the products AC and BD are defined.

(ii) $(A \otimes B)^\top = A^\top \otimes B^\top$.

(iii) If A and B are orthogonal ($A^\top A = I$ and $B^\top B = I$), so is $A \otimes B$. (More generally, if $A^\top A = \alpha I$ and $B^\top B = \beta I$, then $(A \otimes B)^\top (A \otimes B) = \alpha\beta \cdot I$.)

(iv) $\operatorname{tr}(A \otimes B) = \operatorname{tr}(A) \operatorname{tr}(B)$.

Property (i) is the *mixed-product rule*; it is the algebraic engine behind the recursive structure of the Hadamard matrix (Chapter 5).

Example 1.2.2 (Mixed-product rule)**.** Let $A = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right) = H_1$. Then $(H_1 \otimes H_1)(H_1 \otimes H_1) = (H_1 H_1) \otimes (H_1 H_1) = (2I_2) \otimes (2I_2) = 4I_4$. This verifies that $H_2^2 = 4I_4$ (which we will prove in general in Chapter 5).

Example 1.2.3. The Hadamard matrix $H_1 = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$ satisfies

$$H_1 \otimes H_1 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} = H_2.$$

See Section 5.1 for the general construction.

1.3 Probability Essentials

Throughout this guide, probability spaces are finite (typically uniform on $\{-1,+1\}^n$), so measure-theoretic subtleties do not arise.

Definition 1.3.1 (Expectation and variance)**.** For a random variable X on a finite probability space (Ω, μ) (where $\mu(\omega)$ is the probability of outcome ω):

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} \mu(\omega)\, X(\omega), \qquad \mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Theorem 1.3.2 (Markov's inequality)**.** *If $X \geq 0$ and $a > 0$, then $\mathbb{P}[X \geq a] \leq \mathbb{E}[X]/a$.*

Proof. $\mathbb{E}[X] = \sum_\omega \mu(\omega) X(\omega) \geq \sum_{\omega : X(\omega) \geq a} \mu(\omega) \cdot a = a \cdot \mathbb{P}[X \geq a]$. □

Corollary 1.3.3 (Chebyshev's inequality)**.** $\mathbb{P}[|X - \mathbb{E}[X]| \geq t] \leq \mathrm{Var}[X]/t^2$.

Proof. Apply Markov to $(X - \mathbb{E}[X])^2$ with $a = t^2$. □

Markov's inequality is used directly in the thesis to prove the degree-truncation approximation bound (Proposition 3.1 of the thesis).

1.4 The Central Limit Theorem and Berry–Esséen

Theorem 1.4.1 (Central Limit Theorem)**.** *Let $X_1, X_2, \ldots$ be independent, identically distributed random variables with $\mathbb{E}[X_i] = 0$ and $\mathrm{Var}[X_i] = \sigma^2 > 0$. Write $S_n = X_1 + \cdots + X_n$ for the partial sum. Then*

$$\frac{S_n}{\sigma\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1) \quad \text{as } n \to \infty.$$

Here $\xrightarrow{d}$ denotes convergence in distribution*: $\mathbb{P}[S_n/(\sigma\sqrt{n}) \leq t] \to \Phi(t)$ for every t, where $\Phi(t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{t} e^{-s^2/2}\, ds$ is the standard normal CDF.*[2]

[2]For proof via characteristic functions, see Durrett [2019, Chapter 3] or Feller [1971, Chapter XV].

The CLT tells us that $S_n/(\sigma\sqrt{n})$ is approximately $\mathcal{N}(0,1)$ for large n, but says nothing about the rate of convergence. The Berry–Esséen theorem quantifies this.

Theorem 1.4.2 (Berry–Esséen; Berry, 1941, Esséen, 1942)**.** *Under the hypotheses of Theorem 1.4.1, with additionally* $\mathbb{E}[|X_i|^3] = \gamma < \infty$*, we have*

$$\sup_{t\in\mathbb{R}} \left| \mathbb{P}\left[\frac{S_n}{\sigma\sqrt{n}} \le t\right] - \Phi(t) \right| \le \frac{C\gamma}{\sigma^3\sqrt{n}},$$

where C is a universal constant (the best known value is $C < 0.4748$; see Shevtsova, 2011).

Why this matters for the thesis. In the thesis, binary neural network layers compute sums of the form $y_j = \sum_{i=1}^d w_{ji}x_i$ where $w_{ji}, x_i \in \{-1,+1\}$. When x is uniform on $\{-1,+1\}^d$, the summands $w_{ji}x_i$ are independent ± 1 random variables (regardless of the fixed signs w_{ji}), so $\mathbb{E}[w_{ji}x_i] = 0$, $\mathrm{Var}[w_{ji}x_i] = 1$, and $\mathbb{E}[|w_{ji}x_i|^3] = 1$. By Berry–Esséen, $y_j/\sqrt{d}$ is within $O(1/\sqrt{d})$ of $\mathcal{N}(0,1)$ in CDF. At $d = 1024$, this is about 0.015—excellent approximation. This justifies the $1/\sqrt{d}$ normalization used throughout the thesis as a replacement for BitNet's learned scaling factors.

1.5 L^p Norms

For a function f on a probability space (Ω, μ):

Definition 1.5.1 (L^p norm)**.** For $p \ge 1$:

$$\|f\|_p = (\mathbb{E}[|f|^p])^{1/p} = \left(\sum_{\omega\in\Omega} \mu(\omega)\,|f(\omega)|^p\right)^{1/p}.$$

For $p = \infty$: $\|f\|_\infty = \max_\omega |f(\omega)|$.

On a probability space, we always have $\|f\|_p \le \|f\|_q$ for $1 \le p \le q$. To see this: the function $t \mapsto t^{q/p}$ is convex (since $q/p \ge 1$), so by Jensen's inequality (see, e.g., Durrett, 2019, Theorem 1.6.2), $(\mathbb{E}[|f|^p])^{q/p} \le \mathbb{E}[|f|^q]$; taking q-th roots gives $\|f\|_p \le \|f\|_q$. (The direction seems backwards compared to "bigger exponent = bigger norm" for sums, but the normalization by the probability measure reverses things.)

The *hypercontractivity* results in Chapter 4 are striking precisely because they give a reverse inequality $\|T_\rho f\|_q \le \|f\|_p$ for $q > p$, at the cost of applying the noise operator T_ρ.

Remark 1.5.2 (Notation). Throughout this guide, when we write $\mathbb{E}_x[\cdot]$ for functions on $\{-1,+1\}^n$, we mean the expectation under the uniform measure: each $x \in \{-1,+1\}^n$ has probability 2^{-n}. So $\|f\|_2^2 = \mathbb{E}_x[f(x)^2] = 2^{-n}\sum_{x\in\{-1,+1\}^n} f(x)^2$.

Recommended Reading

- **Axler [2024], *Linear Algebra Done Right*, 4th ed.** A clean, determinant-free treatment of inner product spaces and the spectral theorem. Assumes only calculus. Chapters 6–7 cover exactly the inner-product-space machinery used in this guide.
- **Horn and Johnson [2012], *Matrix Analysis*, 2nd ed.** The standard reference for Kronecker products (Chapter 4), matrix norms, and structured matrices. More advanced than Axler; useful when you need precise eigenvalue bounds.
- **Durrett [2019], *Probability: Theory and Examples*, 5th ed.** A graduate-level probability text. Chapters 3–4 cover the CLT, Berry–Esséen, and convergence in distribution rigorously. Freely available from the author's website.
- **Feller [1971], *An Introduction to Probability Theory and Its Applications*, Vol. 2.** The classic reference for characteristic-function proofs of the CLT and Berry–Esséen. Older style but very thorough. Good for seeing where the constant $C < 0.4748$ ultimately comes from.

Chapter 2

Fourier Analysis on the Boolean Hypercube

This chapter develops the Fourier analysis of functions on $\{-1,+1\}^n$ from scratch. The treatment follows O'Donnell [2014], Chapters 1–2, which is the standard reference. A shorter introduction is de Wolf [2008].

2.1 The Boolean Hypercube

Definition 2.1.1 (Boolean hypercube)**.** The *Boolean hypercube* of dimension n is the set

$$\Omega_n = \{-1,+1\}^n = \{(x_1,\ldots,x_n) : x_i \in \{-1,+1\}\}.$$

It has $|\Omega_n| = 2^n$ elements. We equip it with the *uniform probability measure*: each $x \in \Omega_n$ has probability 2^{-n}.

Remark 2.1.2 (Why $\{-1,+1\}$ and not $\{0,1\}$?). Both conventions appear in the literature. The $\{-1,+1\}$ convention is standard in Boolean Fourier analysis because it makes the algebra cleaner:

- Multiplication is the natural group operation ($x_i \cdot x_j \in \{-1,+1\}$), whereas $\{0,1\}$ would require XOR (addition mod 2).
- The Fourier basis functions (parity functions, below) take values in $\{-1,+1\}$, matching the "alphabet" of Boolean functions.
- The connection to the Walsh–Hadamard transform (Chapter 5) is immediate.

To convert: if $b \in \{0,1\}$, set $x = 1 - 2b = (-1)^b$.

Definition 2.1.3 (Boolean and pseudo-Boolean functions)**.** A *Boolean function* is a map $f\colon \{-1,+1\}^n \to \{-1,+1\}$. A *pseudo-Boolean function* is a map $f\colon \{-1,+1\}^n \to \mathbb{R}$.

The set of all pseudo-Boolean functions $f\colon \{-1,+1\}^n \to \mathbb{R}$ forms a real vector space of dimension 2^n (one degree of freedom per point in Ω_n). We equip it with the *expectation inner product*:

Definition 2.1.4 (Inner product on functions)**.** For $f, g\colon \{-1,+1\}^n \to \mathbb{R}$:

$$\langle f, g\rangle = \mathbb{E}_x[f(x)\,g(x)] = \frac{1}{2^n} \sum_{x\in\{-1,+1\}^n} f(x)\,g(x). \tag{2.1}$$

The associated norm is $\|f\|_2 = \sqrt{\langle f, f\rangle} = \sqrt{\mathbb{E}_x[f(x)^2]}$.

Example 2.1.5. If $f\colon \{-1,+1\}^n \to \{-1,+1\}$ is Boolean, then $f(x)^2 = 1$ for all x, so $\|f\|_2^2 = 1$. Every Boolean function is a unit vector in this inner product space.

2.2 The Parity Functions

We now construct an orthonormal basis for the space of pseudo-Boolean functions. The key objects are the *parity functions* (also called *Walsh functions* [Walsh, 1923] or *characters*; see Section 6.3.2 for the group-theoretic viewpoint).

Definition 2.2.1 (Parity function)**.** For each subset $S \subseteq [n] = \{1, 2, \ldots, n\}$, define

$$\chi_S(x) = \prod_{i\in S} x_i. \tag{2.2}$$

By convention, $\chi_\emptyset(x) = 1$ for all x (the empty product).

Since each $x_i \in \{-1,+1\}$, each $\chi_S(x)$ is a product of ± 1 values, hence $\chi_S(x) \in \{-1,+1\}$. The parity functions are themselves Boolean functions. This is a distinctive feature of the Boolean Fourier basis: the basis elements live in the same alphabet as the functions being analyzed.

Remark 2.2.2 (Analogy with classical Fourier analysis). In classical Fourier analysis, the functions $\{e^{2\pi i k x}\}_{k\in\mathbb{Z}}$ form an orthonormal basis for $L^2([0,1])$, and every square-integrable function decomposes into "frequencies" indexed by integers k. Here, the parity functions $\{\chi_S\}_{S\subseteq[n]}$ play the same role: every function on $\{-1,+1\}^n$ decomposes into components indexed by subsets S. (For more on this analogy, see O'Donnell, 2014, §1.1.) The *degree* $|S|$ is the analogue of frequency: degree-0 is the DC component (the mean), degree-1 terms vary linearly, and high-degree terms oscillate rapidly across the hypercube. The key simplification is that the Boolean basis is real-valued (± 1), so there are no complex numbers.

Example 2.2.3. For $n = 3$:

$$\begin{aligned}
\chi_\emptyset(x) &= 1, \\
\chi_{\{1\}}(x) &= x_1, \quad \chi_{\{2\}}(x) = x_2, \quad \chi_{\{3\}}(x) = x_3, \\
\chi_{\{1,2\}}(x) &= x_1x_2, \quad \chi_{\{1,3\}}(x) = x_1x_3, \quad \chi_{\{2,3\}}(x) = x_2x_3, \\
\chi_{\{1,2,3\}}(x) &= x_1x_2x_3.
\end{aligned}$$

There are $2^3 = 8$ such functions, one for each subset of $[3]$.

2.3 Orthonormality

Theorem 2.3.1 (Orthonormality of parity functions)**.** *For any $S, T \subseteq [n]$:*

$$\langle \chi_S, \chi_T \rangle = \delta_{ST} = \begin{cases} 1 & \text{if } S = T, \\ 0 & \text{if } S \neq T. \end{cases} \tag{2.3}$$

Proof. We compute:

$$\langle \chi_S, \chi_T \rangle = \mathbb{E}_x \left[\prod_{i \in S} x_i \cdot \prod_{j \in T} x_j \right] = \mathbb{E}_x \left[\prod_{i \in S \triangle T} x_i \right],$$

where $S \triangle T = (S \setminus T) \cup (T \setminus S)$ is the symmetric difference (we used $x_i^2 = 1$ to cancel shared indices).

If $S = T$, then $S \triangle T = \emptyset$, the product is 1, and the expectation is 1.

If $S \neq T$, then $S \triangle T \neq \emptyset$; pick any $k \in S \triangle T$. Since x_k is independent of $\{x_i : i \neq k\}$ and takes ± 1 with equal probability:

$$\mathbb{E}_x \left[\prod_{i \in S \triangle T} x_i \right] = \mathbb{E}_{x_k}[x_k] \cdot \mathbb{E}_{x_{-k}} \left[\prod_{\substack{i \in S \triangle T \\ i \neq k}} x_i \right] = 0 \cdot (\text{something}) = 0.$$

(Here x_{-k} denotes the coordinates other than k.) □

Since there are 2^n parity functions and the space of pseudo-Boolean functions has dimension 2^n, orthonormality implies:

Corollary 2.3.2. *$\{\chi_S\}_{S \subseteq [n]}$ is an orthonormal basis for the space of all functions $f \colon \{-1, +1\}^n \to \mathbb{R}$ under the inner product $\langle f, g \rangle = \mathbb{E}_x[f(x)g(x)]$.*

2.4 The Fourier Expansion

By Proposition 1.1.4 (general Fourier expansion) and Corollary 2.3.2 (the parity functions form an ONB), we immediately obtain:

Theorem 2.4.1 (Fourier expansion on $\{-1,+1\}^n$)**.** *Every function $f\colon \{-1,+1\}^n \to \mathbb{R}$ has a unique expansion*

$$f(x) = \sum_{S\subseteq[n]} \widehat{f}(S)\,\chi_S(x), \tag{2.4}$$

where the Fourier coefficients *(or* Walsh–Fourier coefficients*) are*

$$\widehat{f}(S) = \langle f, \chi_S\rangle = \mathbb{E}_x\left[f(x)\prod_{i\in S} x_i\right] = \frac{1}{2^n}\sum_{x\in\{-1,+1\}^n} f(x)\prod_{i\in S} x_i. \tag{2.5}$$

The expansion (2.4) is a *multilinear polynomial*: each variable x_i appears with degree at most 1 (because $x_i^2 = 1$ for $x_i \in \{-1,+1\}$, so any higher power reduces). This is the unique multilinear representation of f.

Definition 2.4.2 (Degree)**.** The *degree* of the monomial χ_S is $|S|$. We say f has *Fourier degree at most d* if $\widehat{f}(S) = 0$ for all S with $|S| > d$.

2.5 Parseval's Identity

Applying Proposition 1.1.4 (Parseval in the abstract setting) with the parity ONB:

Theorem 2.5.1 (Parseval's identity)**.** *For any $f\colon \{-1,+1\}^n \to \mathbb{R}$:*

$$\|f\|_2^2 = \mathbb{E}_x[f(x)^2] = \sum_{S\subseteq[n]} \widehat{f}(S)^2. \tag{2.6}$$

For Boolean functions ($f\colon \{-1,+1\}^n \to \{-1,+1\}$):

$$\sum_{S\subseteq[n]} \widehat{f}(S)^2 = 1. \tag{2.7}$$

This is the *energy constraint*: the Fourier coefficients of a Boolean function lie on the unit sphere in $\mathbb{R}^{2^n}$. The "spectral energy" $\widehat{f}(S)^2$ at each subset S can be thought of as the fraction of the function's total variance explained by that frequency component.

Remark 2.5.2 (Relevance to the thesis). The thesis uses Parseval's identity in a crucial way: it implies that the Fourier coefficients of any Boolean function are automatically normalized ($\sum_S \widehat{f}(S)^2 = 1$). This means the spectral parameterization of binary weight matrices comes with a built-in energy budget, replacing the learned per-layer scaling factors α used in BitNet.

2.6 Worked Examples

We compute the Fourier expansion for several important Boolean functions. These examples build intuition for "spectral concentration"—the phenomenon that many natural Boolean functions have most of their Fourier mass on low-degree coefficients.

2.6.1 Dictator Functions

Example 2.6.1 (Dictator)**.** The i-th *dictator function* is $f(x) = x_i$. Its Fourier expansion is trivially $f = \chi_{\{i\}}$, so

$$\widehat{f}(S) = \begin{cases} 1 & \text{if } S = \{i\}, \\ 0 & \text{otherwise.} \end{cases}$$

All spectral energy is at degree 1. Parseval: $1^2 = 1$. ✓

Dictator functions are the extreme case of spectral concentration: all energy is on a single degree-1 coefficient.

2.6.2 The AND Function

Example 2.6.2 (AND on 2 bits)**.** Define $\mathrm{AND}(x_1, x_2) = +1$ iff $x_1 = x_2 = +1$, and -1 otherwise. The truth table:

x_1	x_2	AND
-1	-1	-1
-1	$+1$	-1
$+1$	-1	-1
$+1$	$+1$	$+1$

Computing coefficients using (2.5), $\widehat{f}(S) = \frac{1}{4}\sum_x f(x)\,\chi_S(x)$:

$$\begin{aligned}
\widehat{f}(\emptyset) &= \tfrac{1}{4}[(-1) + (-1) + (-1) + (+1)] = -\tfrac{1}{2}, \\
\widehat{f}(\{1\}) &= \tfrac{1}{4}[(-1)(-1) + (-1)(-1) + (-1)(+1) + (+1)(+1)] = \tfrac{1}{4}[1 + 1 - 1 + 1] = \tfrac{1}{2}, \\
\widehat{f}(\{2\}) &= \tfrac{1}{4}[(-1)(-1) + (-1)(+1) + (-1)(-1) + (+1)(+1)] = \tfrac{1}{4}[1 - 1 + 1 + 1] = \tfrac{1}{2}, \\
\widehat{f}(\{1,2\}) &= \tfrac{1}{4}[(-1)(+1) + (-1)(-1) + (-1)(-1) + (+1)(+1)] = \tfrac{1}{4}[-1 + 1 + 1 + 1] = \tfrac{1}{2}.
\end{aligned}$$

So: $\mathrm{AND}(x_1, x_2) = -\frac{1}{2} + \frac{1}{2}x_1 + \frac{1}{2}x_2 + \frac{1}{2}x_1x_2$.
Parseval: $\frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4} = 1$. ✓
The spectral energy is spread evenly across degrees 0, 1, 1, 2.[1]

[1] See companion notebook, §1: runnable verification of AND and Maj_3 coefficients, Parseval checks, degree truncation, and the convolution theorem.

2.6.3 The Majority Function

Example 2.6.3 (Majority on 3 bits)**.** $\mathrm{Maj}_3(x) = \mathrm{sgn}(x_1 + x_2 + x_3)$, where $\mathrm{sgn}(z) = +1$ if $z > 0$ and $\mathrm{sgn}(z) = -1$ if $z < 0$ (for odd n, the sum $x_1 + \cdots + x_n$ is never zero, so $\mathrm{sgn}(0)$ does not arise). This outputs $+1$ if at least 2 of 3 inputs are $+1$.

Truth table and Fourier computation:

x_1	x_2	x_3	Maj_3
-1	-1	-1	-1
-1	-1	$+1$	-1
-1	$+1$	-1	-1
-1	$+1$	$+1$	$+1$
$+1$	-1	-1	-1
$+1$	-1	$+1$	$+1$
$+1$	$+1$	-1	$+1$
$+1$	$+1$	$+1$	$+1$

The Fourier coefficients are computed by the same procedure as the AND example (applying (2.5) to each of the 8 rows):

$$\begin{aligned}
\widehat{f}(\emptyset) &= 0 \quad \text{(balanced: 4 outputs } +1\text{, 4 outputs } -1\text{)}, \\
\widehat{f}(\{1\}) = \widehat{f}(\{2\}) = \widehat{f}(\{3\}) &= \tfrac{1}{2}, \\
\widehat{f}(\{1,2\}) = \widehat{f}(\{1,3\}) = \widehat{f}(\{2,3\}) &= 0, \\
\widehat{f}(\{1,2,3\}) &= -\tfrac{1}{2}.
\end{aligned}$$

Hence: $\mathrm{Maj}_3(x) = \frac{1}{2}(x_1 + x_2 + x_3) - \frac{1}{2}x_1x_2x_3$.

Parseval: $3 \cdot \frac{1}{4} + \frac{1}{4} = 1$. ✓

The spectral energy breakdown by degree:

Degree	Energy	Fraction
0	0	0%
1	3/4	75%
2	0	0%
3	1/4	25%

Maj_3 is *spectrally concentrated*: 75% of its energy is at degree 1. This is a general phenomenon: the majority function on n bits has most of its spectral energy on degree-1 coefficients, with the concentration improving as n grows [O'Donnell, 2014, Chapter 5].

2.6.4 The Parity Function (Anti-Concentration)

Example 2.6.4 (Parity)**.** $f(x) = x_1x_2 \cdots x_n = \chi_{[n]}(x)$.

This function has $\widehat{f}([n]) = 1$ and all other coefficients zero. All energy is at the maximum degree n. The parity function is the *opposite* of spectrally concentrated: it is maximally complex in the Fourier sense. It is also the hardest Boolean function to learn from random examples [Linial et al., 1993].

2.7 Degree-Level Energy Decomposition

Parseval allows us to decompose the "energy" of a Boolean function by degree:

Definition 2.7.1 (Spectral energy at degree k)**.** For $f\colon \{-1,+1\}^n \to \mathbb{R}$ and $0 \le k \le n$:

$$W^k[f] = \sum_{\substack{S\subseteq[n]\\|S|=k}} \widehat{f}(S)^2.$$

The *spectral energy profile* is the vector $(W^0[f], W^1[f], \ldots, W^n[f])$.

By Parseval: $\sum_{k=0}^{n} W^k[f] = \|f\|_2^2$. For Boolean f, this sum is 1, so the energy profile is a probability distribution on degrees $\{0, 1, \ldots, n\}$.

For a *random* Boolean function (chosen uniformly at random from all 2^{2^n} Boolean functions on n bits), the expected energy at degree k is $\binom{n}{k}/2^n$. To see this: for a random Boolean f, $\mathbb{E}[\widehat{f}(S)^2] = \mathbb{E}[\mathbb{E}_x[f(x)\chi_S(x)]^2]$. By expanding the square and using independence of $f(x)$ and $f(x')$ for $x \neq x'$, one obtains $\mathbb{E}[\widehat{f}(S)^2] = 1/2^n$ for every $S \neq \emptyset$. Since there are $\binom{n}{k}$ subsets of size k, the expected energy at degree k is $\binom{n}{k}/2^n$, which follows a Binomial$(n, 1/2)$ distribution peaked at $k = n/2$. So a random Boolean function has its energy concentrated around degree $n/2$. (See O'Donnell, 2014, Exercise 1.19 for this calculation.)

The thesis's central hypothesis is that *trained* binary weight matrices, viewed as Boolean functions, have their energy concentrated at much lower degrees—far below the $n/2$ baseline of random functions.

2.8 Low-Degree Truncation

Definition 2.8.1 (Degree-k truncation)**.** For $f\colon \{-1,+1\}^n \to \mathbb{R}$:

$$f^{\le k}(x) = \sum_{\substack{S\subseteq[n]\\|S|\le k}} \widehat{f}(S)\,\chi_S(x).$$

Similarly, $f^{>k} = f - f^{\le k}$.

The truncation $f^{\le k}$ is the best degree-k approximation to f in the L^2 norm—this is a standard consequence of orthogonal projection in Hilbert spaces (see Axler, 2024,

Chapter 6): the "tail" $f^{>k}$ is orthogonal to all degree-$\leq k$ functions. The approximation error is:

$$\|f - f^{\leq k}\|_2^2 = \|f^{>k}\|_2^2 = \sum_{|S|>k} \widehat{f}(S)^2. \tag{2.8}$$

For Boolean functions, if we want to *round* $f^{\leq k}$ back to $\{-1,+1\}$ via the sign function, the fraction of inputs where the rounded version disagrees with f is bounded by:

Proposition 2.8.2 (Sign-rounding error; thesis Proposition 3.1)**.** *Let $f\colon \{-1,+1\}^n \to \{-1,+1\}$ be Boolean and $\tilde{f} = \operatorname{sgn}(f^{\leq k})$. Then*

$$\mathbb{P}_x[\tilde{f}(x) \neq f(x)] \leq \sum_{|S|>k} \widehat{f}(S)^2.$$

Proof. If $\tilde{f}(x) \neq f(x)$, then $f(x) \cdot f^{\leq k}(x) \leq 0$. Write $f^{\leq k}(x) = f(x) - f^{>k}(x)$, so $f(x) \cdot f^{\leq k}(x) = 1 - f(x) \cdot f^{>k}(x)$. Thus the disagreement requires $f(x) \cdot f^{>k}(x) \geq 1$, which implies $|f^{>k}(x)| \geq 1$. By Markov's inequality (Theorem 1.3.2) applied to $(f^{>k})^2$:

$$\mathbb{P}[|f^{>k}(x)| \geq 1] \leq \mathbb{E}[(f^{>k}(x))^2] = \sum_{|S|>k} \widehat{f}(S)^2. \qquad \square$$

This bound is tight when $f^{>k}$ is small everywhere it is nonzero (as happens for spectrally concentrated functions), and loose when $f^{>k}$ takes large values on a small set.

Example 2.8.3 (Truncation of Maj_3)**.** For Maj_3, the tail energy above degree 1 is $\widehat{f}(\{1,2,3\})^2 = 1/4$, so the bound gives $\mathbb{P}[\operatorname{sgn}(f^{\leq 1}) \neq f] \leq 1/4$. In fact, $f^{\leq 1}(x) = \frac{1}{2}(x_1 + x_2 + x_3)$, which has the same sign as Maj_3 on all 8 inputs (since $|x_1+x_2+x_3| \geq 1$ always), so the actual error is $0/8 = 0$. The bound is not tight here because $f^{>1}(x) = -\frac{1}{2}x_1x_2x_3$ never has magnitude exceeding $\frac{1}{2} < 1$.

2.9 Plancherel and the Convolution Theorem

Two additional Fourier identities that are used in the thesis:

Theorem 2.9.1 (Plancherel's identity)**.** *For $f, g\colon \{-1,+1\}^n \to \mathbb{R}$:*

$$\langle f, g\rangle = \sum_{S\subseteq[n]} \widehat{f}(S)\,\widehat{g}(S).$$

Proof. Expand both functions: $\langle f, g\rangle = \mathbb{E}_x\left[\sum_S \widehat{f}(S)\chi_S(x) \cdot \sum_T \widehat{g}(T)\chi_T(x)\right] = \sum_S \sum_T \widehat{f}(S)\,\widehat{g}(T)\,\mathbb{E}_x[\chi_S$ $\sum_S \widehat{f}(S)\,\widehat{g}(S)$, where the last step uses orthonormality: $\mathbb{E}[\chi_S\chi_T] = \delta_{ST}$. $\square$

Definition 2.9.2 (Dyadic convolution)**.** For $f, g\colon \{-1,+1\}^n \to \mathbb{R}$, the *dyadic convolution* is

$$(f * g)(x) = \mathbb{E}_y[f(y)\, g(x \cdot y)],$$

where $x \cdot y = (x_1 y_1, \ldots, x_n y_n)$ is coordinatewise multiplication. (The name "dyadic" refers to the group $\mathbb{Z}_2^n$; see Section 6.3.)

Theorem 2.9.3 (Convolution theorem)**.** $\widehat{f * g}(S) = \widehat{f}(S) \cdot \widehat{g}(S)$.

Proof.

$$\begin{aligned}\widehat{f * g}(S) &= \mathbb{E}_x[(f * g)(x)\, \chi_S(x)] = \mathbb{E}_x[\mathbb{E}_y[f(y)\, g(x \cdot y)]\, \chi_S(x)] \\ &= \mathbb{E}_y[f(y)\, \mathbb{E}_x[g(x \cdot y)\, \chi_S(x)]]\,.\end{aligned}$$

Substituting $z = x \cdot y$ (which is uniform when x is uniform, since y is fixed and coordinatewise multiplication by $y \in \{-1,+1\}^n$ is a bijection on $\{-1,+1\}^n$):

$$\mathbb{E}_x[g(x \cdot y)\, \chi_S(x)] = \mathbb{E}_z[g(z)\, \chi_S(z \cdot y)] = \mathbb{E}_z[g(z)\, \chi_S(z)\, \chi_S(y)] = \widehat{g}(S)\, \chi_S(y),$$

using $\chi_S(z \cdot y) = \chi_S(z)\, \chi_S(y)$ (since χ_S is multiplicative: $\prod_{i \in S}(z_i y_i) = \prod_{i \in S} z_i \cdot \prod_{i \in S} y_i$).

Therefore: $\widehat{f * g}(S) = \widehat{g}(S)\, \mathbb{E}_y[f(y)\, \chi_S(y)] = \widehat{g}(S)\, \widehat{f}(S)$. □

Recommended Reading

- **O'Donnell [2014], *Analysis of Boolean Functions.*** The definitive textbook. Chapters 1–2 cover everything in this chapter plus exercises and historical notes. Free preprint on arXiv (2105.10386). Graduate level; assumes real analysis and basic probability.

- **de Wolf [2008], "A Brief Introduction to Fourier Analysis on the Boolean Cube."** A 30-page survey covering the core definitions, Parseval, influences, and a few applications. Excellent for a first pass before reading O'Donnell. Freely available online. Assumes undergraduate linear algebra.

- **Mansour [1994], "Learning Boolean Functions via the Fourier Transform."** An early survey emphasizing the learning-theory applications of Boolean Fourier analysis. Good for seeing how Fourier coefficients connect to PAC learning. Assumes familiarity with basic computational complexity.

Chapter 3

Influence and Noise Sensitivity

This chapter introduces three related concepts—influence, noise sensitivity, and the noise operator—that quantify how "sensitive" a Boolean function is to perturbations of its input. Each has a clean spectral characterization, linking combinatorial/probabilistic notions to the Fourier coefficients. The main reference is O'Donnell [2014, Chapters 2–3].

3.1 Influence of Variables

Informally, the *influence* of variable i measures how often flipping x_i changes the output of f [O'Donnell, 2014, §2.2].

Definition 3.1.1 (Influence)**.** For $f\colon \{-1,+1\}^n \to \{-1,+1\}$ and $i \in [n]$, the *influence of variable* i is

$$\mathrm{Inf}_i[f] = \mathbb{P}_x[f(x) \neq f(x^{\oplus i})], \tag{3.1}$$

where $x^{\oplus i}$ denotes x with the i-th coordinate flipped: $(x_1, \ldots, -x_i, \ldots, x_n)$.[1]

Theorem 3.1.2 (Spectral formula for influence)**.**

$$\mathrm{Inf}_i[f] = \sum_{S \ni i} \widehat{f}(S)^2. \tag{3.2}$$

Proof. Define $D_i f(x) = \frac{1}{2}(f(x) - f(x^{\oplus i}))$, the *discrete derivative* in direction i. Since f takes values in $\{+1,-1\}$, the difference $f(x) - f(x^{\oplus i})$ is either 0 (when they agree) or ± 2 (when they disagree). Thus $D_i f(x) \in \{0, \pm 1\}$, and $(D_i f(x))^2$ is the indicator of the event $f(x) \neq f(x^{\oplus i})$. Therefore:

$$\mathrm{Inf}_i[f] = \mathbb{E}_x[(D_i f(x))^2].$$

[1] See companion notebook, §2: influence computation (combinatorial vs. spectral), the noise operator T_ρ with Monte Carlo verification, and noise stability.

Now compute the Fourier expansion of $D_i f$. We use the *Iverson bracket* notation: $[P]$ equals 1 if predicate P is true and 0 otherwise. Since $\chi_S(x^{\oplus i}) = (-1)^{[i \in S]}\chi_S(x)$ (flipping x_i negates χ_S iff $i \in S$):

$$D_i f(x) = \tfrac{1}{2} \sum_S \widehat{f}(S)\, \chi_S(x)\, (1 - (-1)^{[i \in S]}) = \sum_{S \ni i} \widehat{f}(S)\, \chi_S(x).$$

By Parseval: $\mathbb{E}[(D_i f)^2] = \sum_{S \ni i} \widehat{f}(S)^2$. □

Definition 3.1.3 (Total influence)**.**

$$\mathbf{I}[f] = \sum_{i=1}^{n} \mathrm{Inf}_i[f] = \sum_{S \subseteq [n]} |S| \cdot \widehat{f}(S)^2. \tag{3.3}$$

The total influence is the expected "degree" under the spectral distribution $\{\widehat{f}(S)^2\}_{S \subseteq [n]}$. Functions with low-degree spectral concentration necessarily have low total influence, and conversely.

Example 3.1.4 (Influences of example functions)**.**

- **Dictator** $f(x) = x_1$**:** $\mathrm{Inf}_1[f] = 1$, $\mathrm{Inf}_i[f] = 0$ for $i > 1$. Total: $\mathbf{I}[f] = 1$.
- **Parity** $f(x) = x_1 \cdots x_n$**:** $\mathrm{Inf}_i[f] = 1$ for all i. Total: $\mathbf{I}[f] = n$.
- $\mathbf{Maj}_3$**:** $\mathrm{Inf}_i[f] = \widehat{f}(\{i\})^2 + \widehat{f}(\{1,2,3\})^2 = 1/4 + 1/4 = 1/2$ for each i. Total: $\mathbf{I}[f] = 3/2$.

The total influence is bounded: $\mathbf{I}[f] \le n$ for any Boolean f (since each $\mathrm{Inf}_i[f] \le 1$). The maximum is achieved by the parity function. A celebrated theorem of Kahn et al. [1988] shows that every Boolean function depending on all n variables has $\max_i \mathrm{Inf}_i[f] \ge \Omega(\log n/n)$. Informally, this means no Boolean function can spread its dependence so evenly that *every* variable has negligibly small influence: there is always at least one variable with influence $\Omega(\log n/n)$.

3.2 The Noise Operator

Definition 3.2.1 (Noise operator)**.** For $\rho \in [-1, 1]$, the *Bonami–Beckner noise operator*[2] T_ρ acts on $f\colon \{-1,+1\}^n \to \mathbb{R}$ by

$$T_\rho f(x) = \mathbb{E}_y[f(y)], \tag{3.4}$$

[2]Named after Bonami [1970] and Beckner [1975], who proved the hypercontractivity inequality (Theorem 4.1.1) for this operator.

where y is the "ρ-correlated copy" of x: given a fixed $x \in \{-1,+1\}^n$, each coordinate y_i is chosen independently to equal x_i with probability $\frac{1+\rho}{2}$ and $-x_i$ with probability $\frac{1-\rho}{2}$. The expectation is over the random choice of y (not over x). Thus $T_\rho f$ is a function: it takes a specific x and returns the expected value of f on a random neighbor of x.

Equivalently, $y_i = x_i \cdot z_i$ where z_i is an independent ± 1 random variable with $\mathbb{E}[z_i] = \rho$ (so $\mathbb{P}[z_i = +1] = (1+\rho)/2$ and $\mathbb{P}[z_i = -1] = (1-\rho)/2$).

The crucial property of the noise operator is its diagonal action in the Fourier basis:

Theorem 3.2.2 (Spectral characterization of T_ρ)**.**

$$T_\rho f(x) = \sum_{S \subseteq [n]} \rho^{|S|} \widehat{f}(S)\, \chi_S(x). \tag{3.5}$$

That is, T_ρ multiplies each Fourier coefficient of degree k by ρ^k.

Proof. It suffices to check on the basis elements χ_S:

$$T_\rho \chi_S(x) = \mathbb{E}_y\left[\prod_{i \in S} y_i\right] = \mathbb{E}_y\left[\prod_{i \in S} x_i z_i\right] = \chi_S(x) \cdot \prod_{i \in S} \mathbb{E}[z_i] = \rho^{|S|}\, \chi_S(x).$$

(The z_i are independent, so the expectation factors.) □

Remark 3.2.3 (Interpretation). For ρ close to 1, T_ρ barely changes f: it slightly damps high-degree coefficients. For $\rho = 0$, $T_\rho f(x) = \widehat{f}(\emptyset)$ is just the mean. For ρ close to 0, only the low-degree part of f survives. The noise operator is a "low-pass filter" on the Boolean hypercube. The definition allows negative ρ: at the extreme $\rho = -1$, each $y_i = -x_i$ always, so $T_{-1} f(x) = f(-x)$.

Remark 3.2.4 (Semigroup property). $T_\rho T_\sigma = T_{\rho\sigma}$. This is immediate from Theorem 3.2.2: applying T_ρ then T_σ multiplies each degree-k coefficient by $\rho^k \cdot \sigma^k = (\rho\sigma)^k$. In other words, adding noise twice (with parameters ρ and σ) is the same as adding noise once with parameter $\rho\sigma$. This property is used implicitly in Chapter 4.

Example 3.2.5 (Worked computation: $T_{1/2}\,\mathrm{Maj}_3$)**.** The Fourier expansion of Maj_3 is (see Example 2.6.3):

$$\mathrm{Maj}_3(x) = \tfrac{1}{2}(x_1 + x_2 + x_3) - \tfrac{1}{2}\, x_1 x_2 x_3.$$

Applying $T_{1/2}$ multiplies each degree-k coefficient by $(1/2)^k$:

$$T_{1/2}\,\mathrm{Maj}_3(x) = \tfrac{1}{2} \cdot \tfrac{1}{2}(x_1 + x_2 + x_3) - \tfrac{1}{2} \cdot \tfrac{1}{8}\, x_1 x_2 x_3 = \tfrac{1}{4}(x_1 + x_2 + x_3) - \tfrac{1}{16}\, x_1 x_2 x_3.$$

Sanity check via the probabilistic definition. Take $x = (+1,+1,+1)$, so $\mathrm{Maj}_3(x) = +1$. At $\rho = 1/2$, each $y_i = x_i$ with probability $3/4$ and $y_i = -x_i$ with probability $1/4$.

The only way $\mathrm{Maj}_3(y) = -1$ is if at least two coordinates flip. The probability of exactly j flips is $\binom{3}{j}(1/4)^j(3/4)^{3-j}$:

$$\mathbb{P}[\mathrm{Maj}_3(y) = +1] = \binom{3}{0}(3/4)^3 + \binom{3}{1}(1/4)(3/4)^2 = 27/64 + 27/64 = 54/64,$$
$$\mathbb{P}[\mathrm{Maj}_3(y) = -1] = 10/64.$$

So $T_{1/2}\mathrm{Maj}_3(+1,+1,+1) = (+1)(54/64) + (-1)(10/64) = 44/64 = 11/16$. The spectral formula gives $\frac{1}{4}(1+1+1) - \frac{1}{16}(1) = 3/4 - 1/16 = 11/16$. ✓

3.3 Noise Stability

Definition 3.3.1 (Noise stability)**.** The *noise stability* of f at correlation ρ is

$$\mathrm{Stab}_\rho[f] = \langle f, T_\rho f\rangle = \mathbb{E}_{x,y}[f(x)\, f(y)], \tag{3.6}$$

where (x, y) is a ρ-correlated pair.

Theorem 3.3.2 (Spectral formula)**.**

$$\mathrm{Stab}_\rho[f] = \sum_{S\subseteq[n]} \rho^{|S|}\, \widehat{f}(S)^2. \tag{3.7}$$

Proof. By Plancherel (Theorem 2.9.1): $\langle f, T_\rho f\rangle = \sum_S \widehat{f}(S)\cdot\rho^{|S|}\widehat{f}(S) = \sum_S \rho^{|S|}\widehat{f}(S)^2$. □

Definition 3.3.3 (Noise sensitivity)**.** The *noise sensitivity* is

$$\mathbf{NS}_\delta[f] = \mathbb{P}_{(x,y)}[f(x) \neq f(y)],$$

where y is obtained from x by flipping each bit independently with probability δ, i.e., $\rho = 1 - 2\delta$. For Boolean f:

$$\mathbf{NS}_\delta[f] = \frac{1 - \mathrm{Stab}_{1-2\delta}[f]}{2}.$$

To see why: since $f(x), f(y) \in \{+1, -1\}$, the product $f(x)f(y)$ is $+1$ when they agree and -1 when they disagree, so $\mathbb{P}[f(x) \neq f(y)] = (1 - \mathbb{E}[f(x)f(y)])/2 = (1 - \mathrm{Stab}_\rho[f])/2$. See O'Donnell [2014, §2.4] for further discussion of noise sensitivity and its applications to social choice theory.

Relevance to the thesis. Noise stability quantifies robustness: a layer with high noise stability tolerates occasional bit flips in its weights. The thesis uses noise stability as a

spectral diagnostic and a regularization target (the "stability reward" in Section 4.5 of the thesis encourages low-degree concentration by maximizing $\mathrm{Stab}_\rho[f]$).

3.4 Relationship Between Influence and Noise Sensitivity

Total influence and noise sensitivity are connected:

Proposition 3.4.1. $\mathbf{I}[f] = \lim_{\delta\to 0} \frac{\mathbf{NS}_\delta[f]}{\delta}$.

Proof. $\mathbf{NS}_\delta[f] = \frac{1}{2}\big(1 - \sum_S (1-2\delta)^{|S|}\widehat{f}(S)^2\big)$. Differentiating via the chain rule:

$$\frac{d}{d\delta}\mathbf{NS}_\delta[f] = \frac{1}{2}\sum_S (-1)(-2)|S|\,(1-2\delta)^{|S|-1}\,\widehat{f}(S)^2 = \sum_S |S|\,(1-2\delta)^{|S|-1}\,\widehat{f}(S)^2.$$

Evaluating at $\delta = 0$ (i.e. $1 - 2\delta = 1$) gives $\sum_S |S|\,\widehat{f}(S)^2 = \mathbf{I}[f]$. □

Total influence is the infinitesimal rate of change of noise sensitivity. Functions with high total influence are very sensitive to small perturbations; functions with low total influence are noise-robust.

Example 3.4.2 (Noise stability of example functions).

- **Dictator:** $\mathrm{Stab}_\rho[x_1] = \rho$.
- **Parity:** $\mathrm{Stab}_\rho[x_1 \cdots x_n] = \rho^n \to 0$ exponentially. Parity is extremely noise-sensitive.
- **$\mathbf{Maj}_3$:** Using $\widehat{f}(\emptyset) = 0$, three degree-1 coefficients each of size $\frac{1}{2}$, and one degree-3 coefficient $-\frac{1}{2}$: $\mathrm{Stab}_\rho = 0 \cdot \rho^0 + 3 \cdot \frac{1}{4}\rho + 0 \cdot \rho^2 + \frac{1}{4}\rho^3 = \frac{3\rho+\rho^3}{4}$.

The following table summarises the key quantities for these three functions, illustrating how spectral concentration, influence, and noise sensitivity are tied together.

	Total influence $\mathbf{I}[f]$	$\mathbf{NS}_\delta$ (small δ)	Spectral support
Dictator x_1	1	δ	all at degree 1
Maj_3	3/2	$\frac{3}{2}\delta$	75% at degree 1
Parity $\chi_{[n]}$	n	$n\delta$	all at degree n

Recommended Reading

- **O'Donnell [2014, Chapters 2–3].** The primary source for this chapter. Chapter 2 covers influence and total influence; Chapter 3 covers the noise operator, noise stability, and the connection to social choice theory. Full proofs and exercises.

- **Kahn et al. [1988], "The Influence of Variables on Boolean Functions."** The foundational paper establishing the $\Omega(\log n/n)$ lower bound on maximum influence. Short and elegant. Assumes familiarity with Boolean Fourier basics (i.e., this guide through Chapter 2).

- **Benjamini et al. [1999], "Noise Sensitivity of Boolean Functions and Applications to Percolation."** Introduces noise sensitivity as a concept and connects it to percolation theory and sharp thresholds. Sections 1–3 are accessible with the background from this guide; the percolation applications require additional background.

Chapter 4

Hypercontractivity and Spectral Concentration

This chapter presents two major results: the Bonami–Beckner hypercontractivity inequality and the Linial–Mansour–Nisan (LMN) theorem. Both establish that Boolean functions with bounded "complexity" have their spectral energy concentrated on low-degree coefficients.

The main reference is O'Donnell [2014, Chapters 9–10]. For hypercontractivity specifically, see the original papers [Bonami, 1970, Beckner, 1975] and the survey by O'Donnell [2007]. The closely related logarithmic Sobolev inequality approach is due to Gross [1975].

4.1 The Bonami–Beckner Hypercontractivity Inequality

Hypercontractivity refers to a family of inequalities that bound higher-order norms (L^q) in terms of lower-order norms (L^p) after applying the noise operator. Recall from Section 1.5 that on a probability space, we always have $\|f\|_p \leq \|f\|_q$ for $p \leq q$ (by Jensen). Hypercontractivity gives a *reverse-direction* inequality, at the cost of applying T_ρ.

Theorem 4.1.1 (Bonami–Beckner; Bonami, 1970, Beckner, 1975)**.** *For $f\colon \{-1,+1\}^n \to \mathbb{R}$ and $1 \leq p \leq q$ with $0 \leq \rho \leq \sqrt{(p-1)/(q-1)}$:*

$$\|T_\rho f\|_q \leq \|f\|_p . \tag{4.1}$$

The condition $\rho \leq \sqrt{(p-1)/(q-1)}$ is tight: equality can be attained for any p, q by suitable functions (see O'Donnell, 2014, Theorem 9.21 for the proof of tightness). The most commonly used special case is:

Corollary 4.1.2 ($2 \to q$ hypercontractivity)**.** *For $q \geq 2$ and $\rho \leq 1/\sqrt{q-1}$: $\|T_\rho f\|_q \leq \|f\|_2$.*

Proof of Theorem 4.1.1 for $n = 1$. We prove the base case $n = 1$ to give the flavor of the argument. The general case follows by induction on n, using the product structure of the hypercube. (See O'Donnell [2014, Chapter 9] for details.)

For $n = 1$, any $f\colon \{-1,+1\} \to \mathbb{R}$ can be written $f(x) = a + bx$ with $a = \widehat{f}(\emptyset)$, $b = \widehat{f}(\{1\})$. Then $T_\rho f(x) = a + \rho bx$. We must show $\|a + \rho bx\|_q \le \|a + bx\|_p$.

By homogeneity, we may assume $\|f\|_p = 1$. We need to verify $(|a+\rho b|^q + |a-\rho b|^q)^{1/q} \le 2^{1/q}$ when $(|a+b|^p + |a-b|^p)^{1/p} = 2^{1/p}$.

One approach is to define $F(\rho) = \|a + \rho bx\|_q^q = \frac{1}{2}(|a + \rho b|^q + |a - \rho b|^q)$ and show that $F(\rho) \le F(1)^{q/p}$ whenever $\rho \le \sqrt{(p-1)/(q-1)}$, using convexity of $t \mapsto |t|^q$ and the constraint to bound F''. The condition $\rho \le \sqrt{(p-1)/(q-1)}$ is exactly what controls the curvature. □

Proof for general n (sketch). Write $f(x) = g(x_1, \dots, x_{n-1}) + x_n \cdot h(x_1, \dots, x_{n-1})$ where g and h are functions of $n-1$ variables. The noise operator factors: $T_\rho f = T_\rho g + \rho x_n \cdot T_\rho h$. This is because T_ρ acts independently on each coordinate: the ρ-noise on x_n replaces the x_n factor by ρx_n (the $n = 1$ result), while simultaneously applying T_ρ to g and h in the remaining $n-1$ variables. One then applies the $n = 1$ inequality in the variable x_n (holding the others fixed) and the inductive hypothesis for g and h. □

4.2 The Level-k Inequality

The most important consequence of hypercontractivity for us is the *level-k inequality*, which bounds the Fourier weight at any single degree in terms of the total influence.

Theorem 4.2.1 (Level-k inequality)**.** *For any $f\colon \{-1,+1\}^n \to \{-1,+1\}$ (or more generally $\|f\|_2 \le 1$) and any $k \ge 1$:*

$$W^k[f] = \sum_{|S|=k} \widehat{f}(S)^2 \le \left(\frac{e \cdot \mathbf{I}[f]}{k}\right)^k. \tag{4.2}$$

[1]

Proof. This proof follows O'Donnell [2014, Theorem 9.24]. We first state a key lemma, then use it to prove the inequality.

Bonami Lemma. If $g\colon \{-1,+1\}^n \to \mathbb{R}$ has degree at most k (i.e., $g = g^{\le k}$), then $\|g\|_4 \le 3^{k/2} \|g\|_2$.

Proof of the Bonami Lemma. Set $\rho = 1/\sqrt{3}$, so that the $(p,q) = (2,4)$ case of Bonami–Beckner gives $\|T_\rho h\|_4 \le \|h\|_2$ for any h. Apply this to $h = T_{1/\rho} g$ (here $T_{1/\rho}$ is the

[1] See companion notebook, §3: numerical verification of this bound for Maj_3 and AND_{10}, and LMN k^* computation.

operator that multiplies each degree-j coefficient by $(1/\rho)^j = 3^{j/2}$; it is defined spectrally even though $1/\rho > 1$ places it outside the probabilistic regime of Definition 3.2.1). Since $T_\rho(T_{1/\rho}g) = g$ by the semigroup property (Remark 3.2.4), this gives $\|g\|_4 \leq \left\|T_{1/\rho}g\right\|_2$. Now bound the right side:

$$\left\|T_{1/\rho}g\right\|_2^2 = \sum_{|S|\leq k} (1/\rho)^{2|S|}\,\widehat{g}(S)^2 \leq 3^k\,\|g\|_2^2\,,$$

since $(1/\rho)^{2|S|} = 3^{|S|} \leq 3^k$ for $|S| \leq k$. Taking square roots: $\|g\|_4 \leq 3^{k/2}\,\|g\|_2$. □

Main proof. Define $f^{=k}(x) = \sum_{|S|=k} \widehat{f}(S)\,\chi_S(x)$ (the degree-k component). By Parseval, $W^k[f] = \left\|f^{=k}\right\|_2^2$.

We use a counting/influence argument combined with the noise operator. For any $\rho \in (0,1)$, the noise operator satisfies $(T_\rho f)^{=k} = \rho^k f^{=k}$ (since T_ρ multiplies degree-k coefficients by ρ^k). Therefore:

$$W^k[f] = \rho^{-2k}\left\|(T_\rho f)^{=k}\right\|_2^2 \leq \rho^{-2k}\,\|T_\rho f\|_2^2 = \rho^{-2k}\sum_S \rho^{2|S|}\widehat{f}(S)^2, \tag{4.3}$$

where the inequality holds because $(T_\rho f)^{=k}$ is the orthogonal projection of $T_\rho f$ onto the degree-k subspace, and projecting onto a subspace cannot increase the L^2 norm (this is sometimes called *Bessel's inequality*).

For Boolean f ($\|f\|_\infty = 1$), we use a sharper bound. Each S with $|S| = k$ is counted k times in $\sum_{i=1}^n \sum_{S\ni i,\,|S|=k}$ (once per element of S), so:

$$W^k[f] = \sum_{|S|=k} \widehat{f}(S)^2 = \frac{1}{k}\sum_{i=1}^n \sum_{\substack{S\ni i\\ |S|=k}} \widehat{f}(S)^2 \leq \frac{1}{k}\sum_{i=1}^n \mathrm{Inf}_i[f] = \frac{\mathbf{I}[f]}{k}.$$

This "first moment" bound gives $W^k[f] \leq \mathbf{I}[f]/k$. The exponential improvement comes from combining this counting idea with the Bonami Lemma. The key observation: the first-moment bound is loose because the sum $\sum_{S\ni i}\widehat{f}(S)^2 = \mathrm{Inf}_i[f]$ includes *all* degrees, not just degree k. Hypercontractivity (via the Bonami Lemma) lets us control how much of Inf_i comes from degree exactly k, by bounding the L^4 norm of $f^{=k}$ in terms of its L^2 norm. The full argument, given in O'Donnell [2014, Theorem 9.24], applies this to (4.3) and optimizes the noise parameter. Setting $\rho = \sqrt{k/(e\cdot\mathbf{I}[f])}$ produces the bound

$$W^k[f] \leq \left(\frac{e\,\mathbf{I}[f]}{k}\right)^k. \qquad \square$$

Remark 4.2.2 (Meaning of the level-k inequality). The bound says: if f has total influence I, then the spectral weight at degree k decays super-exponentially once $k > e\cdot I$. For functions with total influence $I = O(1)$ (like dictators), this gives exponential decay

starting from degree 3. For functions with $I = O(\sqrt{n})$ (like majority), the bound becomes nontrivial at $k = O(\sqrt{n})$.

For the thesis application: each row of a binary weight matrix is a Boolean function on $n = \log_2 d$ variables with $\mathbf{I}[f] \leq n$. At $n = 10$ ($d = 1024$), the level-k inequality requires $k > e \cdot 10 \approx 27$ to guarantee decay—well beyond $n = 10$. As the thesis acknowledges (Remark 3.3), the level-k inequality is *vacuous* at this scale. The bound is useful asymptotically but not for the small values of n relevant to the thesis.

Example 4.2.3 (Level-k bound for Maj_3)**.** Maj_3 has total influence $\mathbf{I}[f] = 3/2$ (Example 3.1.4). The level-k inequality gives:

$$W^k[\mathrm{Maj}_3] \leq \left(\frac{e \cdot 3/2}{k}\right)^k.$$

- $k = 1$: bound $= (3e/2)^1 \approx 4.08$. Since $W^1 \leq 1$ always (by Parseval), this is *vacuous*.
- $k = 3$: bound $= (e/2)^3 \approx 2.51$. Still vacuous (exceeds 1).

The actual values are $W^1 = 3/4$ and $W^3 = 1/4$. This illustrates the remark above: for small n, the level-k inequality is asymptotic in character and only becomes useful when $k \gg e \cdot \mathbf{I}[f]$.

For a more favorable case, consider the AND function on 10 bits, $f = \mathrm{AND}_{10}$, which has total influence $\mathbf{I}[f] = 10 \cdot 2^{-(10-1)} = 10/512 \approx 0.0195$. The bound gives $W^k \leq (e \cdot 0.0195/k)^k \approx (0.053/k)^k$, which is less than 10^{-3} already at $k = 1$. Indeed, AND has almost all its mass at degree 0: $\widehat{f}(\emptyset)^2 = (1 - 2^{1-n})^2 \approx 1$.

4.3 The LMN Theorem

The Linial–Mansour–Nisan theorem [Linial et al., 1989, 1993] is one of the foundational results in Boolean Fourier analysis. It establishes that functions computable by small bounded-depth circuits have their spectral energy concentrated on low-degree coefficients.

4.3.1 Background: Boolean Circuits

A *Boolean circuit* is a directed acyclic graph (DAG) where:

- *Input nodes* receive the variables $x_1, \ldots, x_n$ (and their negations).
- *Internal nodes* ("gates") compute AND or OR of their inputs. Each gate has unbounded *fan-in*: it can take any number of inputs. (By contrast, the class NC restricts each gate to fan-in 2; see Arora and Barak, 2009 for standard complexity classes.)

- One designated *output gate* produces the function value.

The *size* M of a circuit is the number of gates. The *depth* D is the length of the longest path from an input to the output. The class AC^0 consists of all families of Boolean functions computable by circuits of polynomial size and *constant* depth.

Example 4.3.1.

- AND, OR, any symmetric function on a constant number of inputs, and any function depending on $O(1)$ variables are in AC^0.
- The majority function Maj_n is *not* in AC^0. It is in TC^0, the class that extends AC^0 by allowing *threshold gates*: a threshold gate outputs 1 if the (weighted) sum of its inputs exceeds a given threshold. This follows from the lower bounds of Razborov [1987].
- The parity function $x_1 \oplus \cdots \oplus x_n$ is *not* in AC^0. This was proved independently by Furst et al. [1984] and Ajtai [1983]. It is consistent with parity having all its spectral energy at degree n.

4.3.2 The Theorem

Theorem 4.3.2 (LMN; Linial et al., 1993)**.** *Let $f\colon \{-1,+1\}^n \to \{-1,+1\}$ be computable by a circuit of depth D and size M. Then for every $k \geq 0$:*

$$\sum_{|S|>k} \widehat{f}(S)^2 \leq 2M \cdot \exp\left(-\frac{k^{1/(D-1)}}{2}\right). \tag{4.4}$$

Remark 4.3.3 (What the LMN theorem says). For a depth-D, size-M circuit:

- The tail energy above degree k decays as $\exp(-k^{1/(D-1)}/2)$, which is super-polynomial in k for constant D.
- Setting the RHS to ϵ and solving for k: the 95% spectral energy is captured by degrees up to $k^* = O((\log M)^{D-1})$. For polynomial-size circuits ($M = n^c$), this gives $k^* = O((\log n)^{D-1})$.
- For depth $D = 2$: the circuits are *DNF* or *CNF* formulas—an OR of ANDs (DNF) or an AND of ORs (CNF)—of size M. Then $k^* = O(\log M)$. Very strong concentration.
- The bound degrades with depth: deeper circuits can "hide" spectral energy at higher degrees.

Proof idea. The proof has two ingredients:

1. Random restrictions. A *random restriction* R sets each variable independently to $+1$, -1, or "$*$" ("leave free") with specified probabilities. The Håstad switching lemma

[Håstad, 1987] provides a quantitative version: if f is computable by a depth-D circuit of size M and each variable is kept alive with probability p, then with probability at least $1 - M \cdot (5p)^{k/(D-1)}$ the restricted function depends on at most k of the surviving variables. Iterating $D - 1$ rounds of restrictions reduces the function to a "junta" (a function depending on at most k of its variables; see O'Donnell, 2014, §3.1) with high probability.

2. Spectral interpretation. A random restriction that keeps each variable alive with probability p and kills it with probability $1 - p$ is equivalent (in the spectral domain) to applying the noise operator $T_{\sqrt{p}}$ [O'Donnell, 2014, §4.1]. The fact that $T_{\sqrt{p}} f$ concentrates on low degrees (after restriction, few variables survive) translates to the tail bound on $\widehat{f}(S)^2$.

The detailed proof is in O'Donnell [2014, Chapter 4] and Linial et al. [1993]. □

Relevance to the thesis. The thesis uses the LMN theorem as motivating evidence for the spectral concentration hypothesis: if trained binary weight matrices correspond to "simple" functions in some complexity-theoretic sense, they should exhibit low-degree spectral concentration. The thesis does *not* claim that trained weights are AC^0 functions—this is an empirical question addressed by the pilot study.

4.4 Learning Fourier-Sparse Functions

The spectral concentration results motivate an algorithmic question: if a Boolean function has its energy on few Fourier coefficients, can we learn it efficiently?

Theorem 4.4.1 (Kushilevitz–Mansour; Kushilevitz and Mansour, 1991)**.** *Let $f\colon \{-1,+1\}^n \to \mathbb{R}$ have an ϵ-concentrated Fourier spectrum, meaning there exists a set $\mathcal{S}$ of at most s subsets such that $\sum_{S \notin \mathcal{S}} \widehat{f}(S)^2 \le \epsilon$. Then f can be approximated to L^2 error ϵ in time* $\mathrm{poly}(n, s, 1/\epsilon)$ *using random examples from the uniform distribution.*

The algorithm works by:

1. Estimating $\widehat{f}(S)$ for "candidate" subsets S, using the formula $\widehat{f}(S) = \mathbb{E}_x[f(x)\,\chi_S(x)]$, which can be estimated from samples.

2. Keeping only the large coefficients.

The challenge is identifying *which* subsets S have large coefficients, since there are 2^n possibilities. Kushilevitz and Mansour showed how to do this efficiently using a "tree pruning" technique: organize the 2^n subsets in a binary tree (one level per variable, branching on whether $i \in S$), and at each node estimate the total squared energy of the subtree. Branches with negligible energy are pruned, so only $O(s)$ branches survive to the leaves, keeping the search polynomial in s rather than exponential in n.

Remark 4.4.2 (Relevance to the thesis). The thesis's spectral parameterization is related in spirit: it restricts the search to degree-$\leq k$ coefficients (of which there are $\binom{n}{\leq k}$), then optimizes them continuously. This is like a continuous relaxation of the KM algorithm, where instead of *estimating* the coefficients from data, we *optimize* them via gradient descent.

Recommended Reading

- **O'Donnell [2014, Chapters 9–10].** Chapter 9 gives the full proof of Bonami–Beckner hypercontractivity, including the two-function version and the level-k inequality. Chapter 10 covers the LMN theorem and random restrictions. Graduate level.

- **O'Donnell [2007], lecture notes.** Earlier, more concise versions of the material that became the textbook. Available online. Good for a quicker read of the hypercontractivity proof.

- **Linial et al. [1993], "Constant Depth Circuits, Fourier Transform, and Learnability."** The original LMN paper. Sections 1–3 contain the main theorem and its proof; Section 4 gives the learning algorithm. Assumes comfort with circuit complexity basics (as introduced in Section 4.3.1).

- **Håstad [1987], *Computational Limitations of Small-Depth Circuits.*** Håstad's PhD thesis, which proved the switching lemma in its strongest form. Technically demanding but historically important—it is the key tool behind both the parity lower bound and the LMN theorem.

- **Kushilevitz and Mansour [1991], "Learning Decision Trees Using the Fourier Spectrum."** The KM algorithm paper. Short and well-written. The tree-pruning idea is explained clearly in Section 3.

Chapter 5

The Walsh–Hadamard Transform

This chapter develops the Walsh–Hadamard Transform (WHT) as the computational realization of the Boolean Fourier expansion. We define Hadamard matrices, establish their properties, describe the fast algorithm (FWHT), and show how the WHT computes Fourier coefficients.

Historical references: Walsh [1923] for Walsh functions, Hadamard [1893] for Hadamard's original construction, Shanks [1969] for the fast algorithm, Beauchamp [1984] and Horadam [2007] for comprehensive treatments.

5.1 Hadamard Matrices

Definition 5.1.1 (Hadamard matrix, Sylvester construction)**.** Define recursively:

$$H_0 = (1), \qquad H_1 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad H_n = H_1 \otimes H_{n-1}, \tag{5.1}$$

where $\otimes$ is the Kronecker product (Section 1.2). The matrix H_n is $2^n \times 2^n$.

Equivalently, $H_n = H_1^{\otimes n}$ (the n-fold Kronecker product of H_1 with itself).

Example 5.1.2.

$$H_2 = H_1 \otimes H_1 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}.$$

Every entry is ± 1.

5.2 Properties of Hadamard Matrices

Theorem 5.2.1 (Properties of H_n)**.** *Let $N = 2^n$. Then:*

(i) All entries are ± 1.

(ii) Symmetry: $H_n = H_n^\top$.

(iii) Orthogonality: $H_n H_n^\top = N \cdot I_N$. *Equivalently,* $H_n^{-1} = \frac{1}{N} H_n$.

(iv) Self-inverse up to scaling: $H_n^2 = N \cdot I_N$.

(v) Rows are parity functions. *Row r of H_n (with r expressed as a subset $S \subseteq [n]$ via its binary representation) is the vector $(\chi_S(x))_{x \in \{-1,+1\}^n}$ (in a specific ordering of the hypercube vertices).*

Proof. (i) follows from the recursive definition: H_1 has ± 1 entries, and the Kronecker product of ± 1 matrices has ± 1 entries.

(ii) Since H_1 is symmetric and $(A \otimes B)^\top = A^\top \otimes B^\top$, $H_n^\top = H_1^\top \otimes H_{n-1}^\top = H_1 \otimes H_{n-1} = H_n$ (by induction).

(iii) By the mixed-product rule for Kronecker products: $H_n H_n^\top = (H_1 \otimes H_{n-1})(H_1^\top \otimes H_{n-1}^\top) = (H_1 H_1^\top) \otimes (H_{n-1} H_{n-1}^\top)$. Since $H_1 H_1^\top = 2I_2$, by induction: $H_n H_n^\top = 2I_2 \otimes 2^{n-1} I_{2^{n-1}} = 2^n I_{2^n} = N \cdot I_N$.

(iv) follows from (ii) and (iii): $H_n^2 = H_n H_n^\top = N \cdot I_N$.

(v) We establish this by identifying each row with a parity function. Number the rows and columns from 0 to $N-1$. For row r and column c, write r and c in binary: $r = (r_{n-1}, \ldots, r_0)$ and $c = (c_{n-1}, \ldots, c_0)$. Then the (r, c) entry of H_n is:

$$(H_n)_{r,c} = (-1)^{\sum_{i=0}^{n-1} r_i c_i} = (-1)^{r \cdot c}, \tag{5.2}$$

where $r \cdot c$ is the bitwise inner product mod 2. (This formula appears, e.g., in Horadam, 2007, Chapter 1.)

Now identify the column index c with the hypercube point $x = ((-1)^{c_0}, (-1)^{c_1}, \ldots, (-1)^{c_{n-1}}) \in \{-1, +1\}^n$, and the row index r with the subset $S = \{i + 1 : r_i = 1\}$. Then:

$$(H_n)_{r,c} = (-1)^{\sum_{i \in S} c_{i-1}} = \prod_{i \in S} (-1)^{c_{i-1}} = \prod_{i \in S} x_i = \chi_S(x).$$

So row r (corresponding to subset S) contains the values of χ_S evaluated at all N hypercube points, in the binary-counting order. □

Remark 5.2.2 (Binary-counting order). The convention above maps the column index c to the hypercube point $x = ((-1)^{c_0}, (-1)^{c_1}, \ldots, (-1)^{c_{n-1}})$, where c_0 is the *least* significant bit. Thus the LSB of c controls the *first* coordinate x_1. For $n = 2$:

c	$(c_1c_0)_2$	$(x_1, x_2) = ((-1)^{c_0}, (-1)^{c_1})$
0	00	$(+1, +1)$
1	01	$(-1, +1)$
2	10	$(+1, -1)$
3	11	$(-1, -1)$

5.3 The Walsh–Hadamard Transform

Definition 5.3.1 (Walsh–Hadamard Transform)**.** For a vector $v \in \mathbb{R}^N$ (with $N = 2^n$), the *Walsh–Hadamard Transform* is

$$\hat{v} = H_n \cdot v. \tag{5.3}$$

The *normalized* WHT is $\hat{v} = \frac{1}{N} H_n \cdot v$. The inverse transform is $v = \frac{1}{N} H_n \cdot \hat{v}$ (since $H_n^{-1} = \frac{1}{N} H_n$).

Theorem 5.3.2 (WHT computes Fourier coefficients)**.** *Let $f\colon \{-1,+1\}^n \to \mathbb{R}$ and let $v \in \mathbb{R}^N$ be the vector of function values in binary-counting order: $v_c = f((-1)^{c_0}, \ldots, (-1)^{c_{n-1}})$ for $c = 0, 1, \ldots, N-1$.*

Then the normalized WHT $\hat{v} = \frac{1}{N} H_n v$ gives the Fourier coefficients:

$$\hat{v}_r = \widehat{f}(S_r),$$

where S_r is the subset corresponding to the binary representation of r.

Proof.

$$\hat{v}_r = \frac{1}{N} \sum_{c=0}^{N-1} (H_n)_{r,c} \cdot v_c = \frac{1}{N} \sum_{x \in \{-1,+1\}^n} \chi_{S_r}(x) \cdot f(x) = \mathbb{E}_x[f(x)\, \chi_{S_r}(x)] = \widehat{f}(S_r).$$

The second equality uses property (v) of Theorem 5.2.1. □

Remark 5.3.3 (Self-inverse property and the thesis). Since $H_n^2 = N \cdot I$, the WHT is (up to scaling) its own inverse. In the thesis, this means:

- *Forward:* Given binary weights $w \in \{-1,+1\}^N$, compute Fourier coefficients via $\hat{w} = \frac{1}{N} H_n w$.
- *Inverse:* Given spectral coefficients $\hat{w}$, reconstruct spatial weights via $w = H_n \hat{w}$ (then apply sign).

Both directions use the same matrix, just with different scaling.

5.4 The Fast Walsh–Hadamard Transform

The naive matrix-vector multiplication $H_n v$ costs $O(N^2)$ operations. The *Fast Walsh–Hadamard Transform* (FWHT) [Shanks, 1969] exploits the recursive Kronecker structure to compute $H_n v$ in $O(N \log N) = O(N \cdot n)$ operations, using only additions and subtractions (no multiplications).

5.4.1 The Butterfly Algorithm

The recursion $H_n = H_1 \otimes H_{n-1}$ yields a divide-and-conquer decomposition. The Kronecker product gives the block form

$$H_1 \otimes H_{n-1} = \begin{pmatrix} 1 \cdot H_{n-1} & 1 \cdot H_{n-1} \\ 1 \cdot H_{n-1} & (-1) \cdot H_{n-1} \end{pmatrix} = \begin{pmatrix} H_{n-1} & H_{n-1} \\ H_{n-1} & -H_{n-1} \end{pmatrix}.$$

Write $v = \binom{v_0}{v_1}$ where $v_0, v_1 \in \mathbb{R}^{N/2}$ are the first and second halves. Then:

$$H_n v = \begin{pmatrix} H_{n-1} & H_{n-1} \\ H_{n-1} & -H_{n-1} \end{pmatrix} \begin{pmatrix} v_0 \\ v_1 \end{pmatrix} = \begin{pmatrix} H_{n-1}(v_0 + v_1) \\ H_{n-1}(v_0 - v_1) \end{pmatrix}.$$

Each of the two recursive calls operates on a vector of half the size. This gives the recurrence $T(N) = 2T(N/2) + O(N)$, solving to $T(N) = O(N \log N)$.

Algorithm 1 Fast Walsh–Hadamard Transform (in-place)

Require: $v \in \mathbb{R}^N$ with $N = 2^n$

```
for ℓ = 0, 1, ..., n − 1 do
    h ← 2^ℓ                                          ▷ half-block size
    for j = 0, 2h, 4h, ..., N − 2h do
        for i = j, j + 1, ..., j + h − 1 do
            a ← v[i]
            b ← v[i + h]
            v[i] ← a + b
            v[i + h] ← a − b
        end for
    end for
end for
return v                                             ▷ v now contains H_n times the original v
```

[1]

[1] See companion notebook, §4: NumPy and PyTorch FWHT implementations, step-by-step trace, and timing comparison against naïve matrix multiplication.

Iterative vs. recursive. The recursive decomposition processes coordinates top-down (splitting on x_n first, then x_{n-1}, etc.), while the iterative Algorithm 1 processes them bottom-up ($\ell = 0, 1, \ldots, n-1$). Both perform the same set of butterfly operations and produce the same result; only the order differs.

Operation count. The triple loop performs exactly $n \cdot N/2$ additions and $n \cdot N/2$ subtractions, for a total of $nN = N \log_2 N$ addition/subtraction operations. *No multiplications are needed.*

Normalization convention. Algorithm 1 computes the *unnormalized* WHT: the output is $H_n v$. To obtain the Fourier coefficients $\widehat{f}(S)$, divide by N (i.e., compute $\frac{1}{N} H_n v$). Some implementations incorporate the $1/N$ factor into the algorithm or split it as $1/\sqrt{N}$ on both the forward and inverse transforms. In this guide, we follow the convention that the forward transform is unnormalized and the inverse includes the $1/N$ factor, matching Definition 5.3.1.

This is the crucial property for the thesis: when the input vector v has integer entries (e.g., $v \in \{-1, +1\}^N$), the entire FWHT can be performed in integer arithmetic. The output entries are integers in the range $[-N, N]$.

Example 5.4.1 (FWHT of a small vector)**.** Let $n = 2$, $N = 4$, and $v = (1, -1, -1, 1)^\top$ (corresponding to the function $f(x_1, x_2) = x_1 x_2$).

Step $\ell = 0$ ($h = 1$):

$$(v[0], v[1]) \leftarrow (1 + (-1),\ 1 - (-1)) = (0, 2),$$
$$(v[2], v[3]) \leftarrow (-1 + 1,\ -1 - 1) = (0, -2).$$

After step 0: $v = (0, 2, 0, -2)$.

Step $\ell = 1$ ($h = 2$):

$$(v[0], v[2]) \leftarrow (0 + 0,\ 0 - 0) = (0, 0),$$
$$(v[1], v[3]) \leftarrow (2 + (-2),\ 2 - (-2)) = (0, 4).$$

After step 1: $v = (0, 0, 0, 4)$.

So $H_2 v = (0, 0, 0, 4)$. The normalized WHT is $\frac{1}{4}(0, 0, 0, 4) = (0, 0, 0, 1)$.

Reading off Fourier coefficients: $\widehat{f}(\emptyset) = 0$, $\widehat{f}(\{1\}) = 0$, $\widehat{f}(\{2\}) = 0$, $\widehat{f}(\{1, 2\}) = 1$. Indeed, $f = x_1 x_2 = \chi_{\{1,2\}}$. ✓

5.5 Comparison with the FFT

The FWHT is the Boolean analogue of the Fast Fourier Transform (FFT) of Cooley and Tukey [1965]. The following table highlights the parallels:

Property	**FFT (classical)**	**FWHT (Boolean)**
Domain	$\mathbb{Z}/N\mathbb{Z}$ or $[0, 2\pi)$	$\{-1, +1\}^n$
Basis	$e^{2\pi ik/N}$	$\chi_S(x) = \prod_{i \in S} x_i$
Basis values	Complex roots of unity	± 1
Matrix	DFT matrix F_N	Hadamard matrix H_n
Self-inverse?	$F_N^{-1} = \frac{1}{N}\overline{F_N}$	$H_n^{-1} = \frac{1}{N}H_n$
Fast algorithm	$O(N \log N)$ complex	$O(N \log N)$ real
Arithmetic	Complex multiply-add	Integer add-subtract
Convolution	Circular convolution	Dyadic convolution

The FWHT is simpler than the FFT: it uses only additions and subtractions (no twiddle factors, no complex arithmetic). This makes it particularly attractive for hardware implementation and for the binary neural network setting of the thesis.

5.6 The Convolution Theorem via the WHT

The convolution theorem (Theorem 2.9.3) becomes a matrix identity in the WHT:

Proposition 5.6.1. *Let $v, w \in \mathbb{R}^N$ represent functions on $\{-1, +1\}^n$. Define the* dyadic convolution *vector $(v \circledast w)_c = \frac{1}{N}\sum_d v_d \cdot w_{c \oplus d}$, where $\oplus$ denotes bitwise XOR (addition mod 2 on each coordinate). Then:*

$$H_n(v \circledast w) = \frac{1}{N}(H_n v) \odot (H_n w),$$

where $\odot$ is componentwise multiplication.

This is Theorem 2.9.3 (the abstract convolution theorem from Chapter 2) written in matrix notation. It means dyadic convolution can be performed in $O(N \log N)$ time: transform both vectors, multiply componentwise, inverse-transform.

5.7 The WHT as a Neural Network Layer

A key observation in the thesis is that the WHT (or its inverse) can serve as a building block for neural network layers.

Hadamard layer. A *Hadamard layer* computes $y = \frac{1}{\sqrt{N}} H_n \cdot \mathrm{diag}(s) \cdot x$, where $s \in \{-1, +1\}^N$ is a learnable sign vector. The sign-flip $\mathrm{diag}(s) \cdot x$ multiplies each component of x by ± 1 (a componentwise sign change). The Hadamard multiplication H_n then mixes all components.

The normalization $1/\sqrt{N}$ ensures that $\|y\|_2 = \|x\|_2$: $\mathrm{diag}(s)$ preserves the norm since $|s_i| = 1$, and $\frac{1}{\sqrt{N}} H_n$ preserves it because $\left\| \frac{1}{\sqrt{N}} H_n x \right\|_2^2 = \frac{1}{N} x^\top H_n^\top H_n x = \frac{1}{N} x^\top (NI) x = \|x\|_2^2$, using property (iii) of Theorem 5.2.1.

Spectral reconstruction. The thesis's `SpectralLinear` layer works differently: it stores only the low-degree spectral coefficients $\hat{w}_{\leq k}$ (degree $\leq k$) and pads the remaining high-degree entries with zeros to form a full N-length vector $\hat{w}$. It then reconstructs the spatial weight vector via $w = H_n \hat{w}$ (using the FWHT) and applies $\mathrm{sgn}(w)$ to get binary weights. The FWHT always operates on a full N-length vector; the truncation is implemented by zero-padding. The reconstruction costs $O(N \log N)$ per row per training step, but is not needed at inference (the binary weights are stored directly).

Since the FWHT involves only additions and subtractions, it is numerically very stable even for floating-point inputs (such as the continuous spectral coefficients produced by gradient descent).

Recommended Reading

- **Beauchamp [1984], *Applications of Walsh and Related Functions.*** A comprehensive treatment of Walsh functions from a signal-processing perspective. Covers sequency ordering, fast algorithms, and hardware implementations. Undergraduate engineering level. Good for readers coming from a signal-processing background rather than pure mathematics.

- **Shanks [1969], "Computation of the Fast Walsh–Fourier Transform."** The original paper describing the butterfly-based FWHT algorithm. Very short (3 pages). Shows the connection to the Cooley–Tukey FFT.

- **Horadam [2007], *Hadamard Matrices and Their Applications.*** A thorough treatment of Hadamard matrices: existence, constructions (Sylvester, Paley, product), and applications in coding theory, combinatorics, and signal processing. Graduate level. Chapters 1–3 cover the material most relevant to this guide.

- **Dao et al. [2022], "Monarch: Expressive Structured Matrices."** Shows how Hadamard-based structured matrices can replace dense layers in deep learning. Relevant for understanding how the FWHT fits into modern efficient-architecture design. Assumes familiarity with deep learning and matrix factorizations.

Chapter 6

Linear Threshold Functions, Codes, and Groups

This chapter covers three topics that connect the Boolean Fourier theory to the thesis: linear threshold functions (the basic computational unit of binary networks), Reed–Muller codes (the coding-theoretic interpretation of spectral truncation), and the group-theoretic viewpoint (which places the Walsh–Fourier basis in a broader context).

6.1 Linear Threshold Functions

Definition 6.1.1 (Linear threshold function (LTF))**.** A *linear threshold function* is a Boolean function of the form

$$f(x) = \operatorname{sgn}\left(\sum_{i=1}^{n} w_i x_i\right) = \operatorname{sgn}(w^\top x), \tag{6.1}$$

where $w = (w_1, \ldots, w_n) \in \mathbb{R}^n$ is a weight vector and $\operatorname{sgn}(z) = +1$ if $z \geq 0$ and -1 if $z < 0$.

LTFs are also called *halfspaces* or *threshold gates*. In the neural network context, a single neuron with binary inputs and a sign activation computes exactly an LTF. A binary neural network layer computes d LTFs in parallel (one per row of the weight matrix).

Example 6.1.2.

- **Dictator:** $f(x) = \operatorname{sgn}(x_1) = x_1$. Weight vector $w = (1, 0, \ldots, 0)$.
- **Majority:** $f(x) = \operatorname{sgn}(x_1 + \cdots + x_n)$. Weight vector $w = (1, 1, \ldots, 1)$.
- **Weighted threshold:** $f(x) = \operatorname{sgn}(3x_1 + x_2 - 2x_3)$. This function outputs $+1$ when $3x_1 + x_2 - 2x_3 \geq 0$.

The parity function $x_1 x_2 \cdots x_n$ is *not* an LTF for $n \geq 2$ (it cannot be computed by a single threshold gate; this classical fact goes back to Minsky and Papert, 1969).

6.1.1 Chow's Theorem

Theorem 6.1.3 (Chow, 1961; Chow, 1961; see also O'Donnell [2014, Chapter 5]). *A linear threshold function f is uniquely determined among all LTFs by its degree-0 and degree-1 Fourier coefficients: $\widehat{f}(\emptyset) = \mathbb{E}[f]$ and $\widehat{f}(\{i\}) = \mathbb{E}[f \cdot x_i]$ for $i = 1, \ldots, n$.*

[1]

The $n+1$ values $(\widehat{f}(\emptyset), \widehat{f}(\{1\}), \ldots, \widehat{f}(\{n\}))$ are called the *Chow parameters*. Chow's theorem says: if two LTFs have the same Chow parameters, they are the same function.

Remark 6.1.4. Chow's theorem implies that the "essential information" of an LTF lives in degree ≤ 1 of the Fourier spectrum. This does not mean the higher-degree coefficients are zero—they are generally nonzero—but that they are *determined* by the low-degree ones (within the class of LTFs). This is a strong structural result supporting the idea that LTFs are "low-complexity" functions.

Proof sketch. Two distinct LTFs $f = \mathrm{sgn}(w^\top x)$ and $g = \mathrm{sgn}(v^\top x)$ define different halfspaces in $\mathbb{R}^n$. If they had the same Chow parameters, then $\mathbb{E}[f \cdot x_i] = \mathbb{E}[g \cdot x_i]$ for all i, meaning f and g agree on average with every coordinate projection.

The key geometric insight: the Chow parameters determine the orthogonal projection of f onto the degree-1 subspace of $L^2(\{-1,+1\}^n)$. For LTFs, this projection is essentially the (normalized) weight vector—one can show $\widehat{f}(\{i\}) = c \cdot w_i / \|w\|_2$ for a constant c depending only on $\|w\|$—so two LTFs with the same Chow parameters must have proportional weight vectors, hence define the same halfspace. See O'Donnell [2014, Chapter 5] for the full argument. □

6.1.2 Spectral Properties of LTFs

LTFs have several spectral properties that are relevant to the thesis:

Spectral concentration. LTFs are spectrally concentrated at low degrees. While the exact spectrum depends on the weight vector w, the majority function (equal weights) has 75% of its energy at degree 1 for $n = 3$, and the concentration improves with n. The total influence bound $\mathbf{I}[f] = O(\sqrt{n})$ (Theorem 6.1.5 below), combined with the level-k inequality (Theorem 4.2.1), gives the tail bound $W^k[f] \leq (e\sqrt{n}/k)^k$, which becomes small when $k \gg e\sqrt{n}$. So most of the spectral energy is within the first $O(\sqrt{n})$ degrees.

Noise sensitivity. LTFs have bounded noise sensitivity. The following result quantifies this:

[1] See companion notebook, §5: Chow parameters of Maj_n for $n = 3, 5, 7$ and a weighted threshold gate.

Theorem 6.1.5 (Harsha et al. [2014]). *For any LTF $f\colon \{-1,+1\}^n \to \{-1,+1\}$, the total influence satisfies $\mathbf{I}[f] = O(\sqrt{n})$.*

More precisely, for $f = \operatorname{sgn}(w^\top x)$ with $\|w\|_2 = 1$: $\mathbf{I}[f] = \Theta(\|w\|_1)$ as $n \to \infty$ (the asymptotic is in n, with the implicit constants being absolute). The idea: each coordinate's influence equals the probability that x_i is "pivotal" (flipping x_i changes the sign of $w^\top x$), which can be estimated by the CLT applied to the sum $w^\top x$ minus the i-th term. Berry–Esséen (Theorem 1.4.2) makes this rigorous. For binary weights $w \in \{-1,+1\}^n$, $\|w\|_1 = n$ and $\|w\|_2 = \sqrt{n}$, giving $\mathbf{I}[f] \approx \sqrt{2n/\pi}$ (see Peres, 2004 for the precise asymptotics).

Relevance to the thesis. Each row of a binary weight matrix followed by a sign activation computes an LTF. Note that here n denotes the number of inputs to the LTF (i.e., the layer width d in the thesis, not $\log_2 d$). The bounded noise sensitivity of LTFs means that:

- Each neuron in a binary layer with d inputs has total influence $O(\sqrt{d})$; at $d = 1024$ this is ≈ 25.
- The spectral tail (high-degree coefficients) is bounded, supporting the spectral concentration hypothesis.
- The sign activation acts as a "spectral filter," preserving low-degree structure and discarding high-frequency noise.

Remark 6.1.6 (Variable naming). Caution: in the LTF literature, n denotes the number of input bits to the function. In the thesis's spectral parameterization, each row of a $d \times d$ weight matrix is viewed as a Boolean function on $n = \log_2 d$ variables (the "Fourier domain" variables). These are different uses of n. When we say an LTF on d inputs has total influence $O(\sqrt{d})$, we mean the LTF viewed as a function on d literal input coordinates, *not* on the $\log_2 d$ Fourier variables.

6.2 Reed–Muller Codes

There is a deep connection between the Boolean Fourier basis and Reed–Muller codes from coding theory. The thesis uses this connection to analyze the spectral-to-binary projection.

References: Reed [1954], Muller [1954] for the original constructions; MacWilliams and Sloane [1977] for a comprehensive treatment.

6.2.1 Definition

Definition 6.2.1 (Reed–Muller code)**.** Here $\mathrm{GF}(2) = \{0, 1\}$ is the *field with two elements*, where addition is XOR ($0 \oplus 1 = 1$, $1 \oplus 1 = 0$) and multiplication is AND ($1 \cdot 1 = 1$, all others 0). A *linear code* of length N over $\mathrm{GF}(2)$ is a linear subspace of $\mathrm{GF}(2)^N$; its *dimension* is the dimension of this subspace, and its *minimum distance* is the minimum *Hamming distance* between distinct codewords (the Hamming distance between two vectors is the number of coordinates in which they differ).

The r-th order *Reed–Muller code* $\mathrm{RM}(r, n)$ consists of all evaluation vectors of polynomials $p(b_1, \ldots, b_n)$ over $\mathrm{GF}(2)$ of total degree $\leq r$, evaluated at all 2^n points of $\{0, 1\}^n$.

GF(2) perspective. $\mathrm{RM}(r, n)$ is a linear code of length 2^n over $\mathrm{GF}(2)$. Its dimension is $\sum_{i=0}^{r} \binom{n}{i}$ and its minimum distance is 2^{n-r}.

$\{-1, +1\}$ perspective. We can convert to the $\{-1, +1\}$ convention via $b \mapsto (-1)^b$. Under this map, a $\mathrm{GF}(2)$-monomial $b_{i_1} \cdots b_{i_k}$ (which computes AND over $\{0, 1\}$) does *not* become the Walsh monomial $\chi_{\{i_1,\ldots,i_k\}}(x) = x_{i_1} \cdots x_{i_k}$. Instead, $b_i = \frac{1-x_i}{2}$, so $\mathrm{GF}(2)$-monomials of degree k become $\{-1, +1\}$-polynomials involving Walsh monomials of degree $\leq k$ (but not only degree k).

Nevertheless, the *span* is preserved: the vector space of $\mathrm{GF}(2)$-polynomials of degree $\leq r$, after the $b \mapsto (-1)^b$ map, coincides with the vector space spanned by $\{\chi_S : |S| \leq r\}$. This is because both are $(r + 1)$-level filtrations of the same 2^n-dimensional space, and the change-of-basis between the two is lower-triangular with nonzero diagonal entries: $b_i = (1 - x_i)/2$, so a $\mathrm{GF}(2)$-monomial of degree k equals $\prod_{i \in S}(1 - x_i)/2^k$, which is $(\pm 1/2^k)\, \chi_S$ plus lower-degree Walsh terms. The leading coefficient is nonzero, ensuring invertibility at each level.

Thus, the real span of the $\{0, 1\}$-evaluation vectors of $\mathrm{RM}(r, n)$ equals the Walsh subspace $\mathrm{span}\{\chi_S : |S| \leq r\}$.

Caution: The $\{0, 1\}$-evaluation vectors lie in the degree-$\leq r$ Walsh subspace, but the $\{-1, +1\}$-codewords obtained via $0 \mapsto +1$, $1 \mapsto -1$ (i.e., $v \mapsto (-1)^v$) can have Walsh–Fourier degree $> r$, because the map $v \mapsto (-1)^v$ is nonlinear. For example, $b_1 \oplus b_2$ has $\mathrm{GF}(2)$-degree 1, but $(-1)^{b_1 \oplus b_2} = (-1)^{b_1}(-1)^{b_2} = x_1 x_2$, which has Walsh degree 2. More strikingly, $\bigoplus_{i=1}^{n} b_i$ (degree 1 over $\mathrm{GF}(2)$) maps to $x_1 \cdots x_n = \chi_{[n]}$, the full parity of Walsh degree n.

Theorem 6.2.2 (RM parameters; MacWilliams and Sloane, 1977, Chapter 13)**.** *$RM(r, n)$ is a linear code with:*

- Length: $N = 2^n$.
- Dimension: $\sum_{i=0}^{r} \binom{n}{i}$.

- Minimum distance: 2^{n-r}.

Example 6.2.3.

- $\mathrm{RM}(0, n)$: repetition code. Only two codewords: all-1 and all-(-1). Minimum distance 2^n. (Degree 0 functions are constants ± 1.)

- $\mathrm{RM}(1, n)$: the first-order RM code. Dimension $n+1$. Codewords are the $\{-1, +1\}$-evaluation vectors of affine functions $c_\emptyset \oplus c_1 b_1 \oplus \cdots \oplus c_n b_n$ over GF(2) (degree ≤ 1 over GF(2)). There are 2^{n+1} codewords. Minimum distance 2^{n-1}.

  Worked example: RM$(1, 2)$.[2] The affine functions over GF(2) in two variables are: 0, 1, b_1, $1 \oplus b_1$, b_2, $1 \oplus b_2$, $b_1 \oplus b_2$, $1 \oplus b_1 \oplus b_2$—eight functions in all ($2^{n+1} = 2^3 = 8$). Their evaluation vectors over $\{0, 1\}^2$, mapped to $\{-1, +1\}$ via $0 \mapsto +1$, $1 \mapsto -1$:

Function	$\{0,1\}^2$ eval	$\{-1,+1\}$ codeword
0	$(0,0,0,0)$	$(+1,+1,+1,+1)$
b_1	$(0,1,0,1)$	$(+1,-1,+1,-1)$
b_2	$(0,0,1,1)$	$(+1,+1,-1,-1)$
$b_1 \oplus b_2$	$(0,1,1,0)$	$(+1,-1,-1,+1)$

  (plus their bitwise complements for the remaining four). The $\{0, 1\}$-evaluation vectors lie in the degree-≤ 1 Walsh subspace $\mathrm{span}\{1, x_1, x_2\}$. Note, however, that the $\{-1, +1\}$-codeword for $b_1 \oplus b_2$ is $(+1, -1, -1, +1) = x_1 x_2$, which has Walsh degree 2 (illustrating the degree-increase from the $v \mapsto (-1)^v$ map). Minimum distance $= 2^{2-1} = 2$.

- $\mathrm{RM}(n, n)$: the full code, containing all 2^{2^n} Boolean functions. Minimum distance 1.

6.2.2 Connection to the Thesis

The thesis's degree-k spectral parameterization optimizes within the span of $\{\chi_S : |S| \leq k\}$—which is exactly the "continuous relaxation" of $\mathrm{RM}(k, n)$. The coefficients are real during training and are projected to $\{-1, +1\}$ via the sign function.

Three properties of RM codes are relevant:

(i) **Minimum distance.** $\mathrm{RM}(k, n)$ has minimum distance 2^{n-k}. For the thesis with $k = 3$ and $n = 10$: minimum distance $= 2^7 = 128$. This means any two distinct degree-≤ 3 Boolean functions differ on at least 128 of the 1024 inputs.

(ii) **Dimension = parameter count.** The dimension $\sum_{i=0}^{k} \binom{n}{i}$ is exactly the number of spectral coefficients in the thesis's parameterization: $\binom{10}{\leq 3} = 1 + 10 + 45 + 120 = 176$.

[2] See companion notebook, §5: $\mathrm{RM}(1, 2)$ codeword construction and the degree-increase phenomenon for $n = 2, \ldots, 6$.

(iii) **Sign projection is not RM decoding.** The sign function applied to a continuous approximation $f^{\leq k}(x)$ is *not* guaranteed to produce a codeword of $\mathrm{RM}(k,n)$. The sign of a degree-k polynomial can have Fourier support up to degree n. For example, the degree-1 polynomial $p(x) = \frac{1}{2}(x_1 + x_2 + x_3)$ has $\mathrm{sgn}(p) = \mathrm{Maj}_3$, which has a nonzero degree-3 Fourier coefficient $\widehat{\mathrm{Maj}_3}(\{1,2,3\}) = -\frac{1}{2}$. The sign operation "creates" high-degree spectral content. The RM connection provides structural insight but does not directly constrain the output.

6.3 The Group-Theoretic Perspective

The Boolean Fourier expansion is a special case of Fourier analysis on finite abelian groups. This section briefly develops that perspective. The main reference is Terras [1999]; see also Diaconis [1988].

6.3.1 The Group $\mathbb{Z}_2^n$

The Boolean hypercube $\{-1,+1\}^n$ under coordinatewise multiplication is isomorphic to the group $\mathbb{Z}_2^n = (\mathbb{Z}/2\mathbb{Z})^n$ under coordinatewise addition mod 2.

The isomorphism: $\phi\colon \mathbb{Z}_2^n \to \{-1,+1\}^n$ by $\phi(b_1,\ldots,b_n) = ((-1)^{b_1},\ldots,(-1)^{b_n})$. Then $\phi(a+b) = \phi(a)\cdot\phi(b)$ (coordinatewise).

6.3.2 Characters of Finite Abelian Groups

Definition 6.3.1 (Character)**.** A *character* of a finite abelian group G is a group homomorphism $\chi\colon G \to \mathbb{C}^*$ (the nonzero complex numbers under multiplication). In general, characters are complex-valued, but for $\mathbb{Z}_2^n$ they turn out to be real—in fact $\{-1,+1\}$-valued—as the proof below shows.

Theorem 6.3.2. *The characters of $\mathbb{Z}_2^n$ are exactly the functions $\chi_S(b) = (-1)^{\sum_{i\in S} b_i}$ for $S \subseteq [n]$. Under the $\{-1,+1\}$ convention, these become $\chi_S(x) = \prod_{i\in S} x_i$: exactly the parity functions.*

Proof. A character χ of $\mathbb{Z}_2^n$ is determined by its values on the standard generators $e_i = (0,\ldots,1,\ldots,0)$. Since $2e_i = 0$ in $\mathbb{Z}_2^n$, $\chi(e_i)^2 = \chi(0) = 1$, so $\chi(e_i) \in \{+1,-1\}$. The 2^n choices for $(\chi(e_1),\ldots,\chi(e_n)) \in \{-1,+1\}^n$ give all 2^n characters. □

Theorem 6.3.3 (Fourier analysis on finite abelian groups)**.** *For any finite abelian group G with character group $\hat{G}$, the characters form an orthonormal basis for $L^2(G)$[3] (functions $G \to \mathbb{C}$ with the inner product $\langle f,g\rangle = \frac{1}{|G|}\sum_{x\in G} f(x)\overline{g(x)}$). Every function $f\colon G \to \mathbb{C}$ has a unique expansion $f = \sum_{\chi\in\hat{G}} \widehat{f}(\chi)\,\chi$, and Parseval holds: $\|f\|_2^2 = \sum_\chi |\widehat{f}(\chi)|^2$.*

[3]Proof: see Terras [1999, Chapters 1–2] or Diaconis [1988, Chapter 1].

For $G = \mathbb{Z}_2^n$, this recovers exactly the Boolean Fourier expansion of Chapter 2.

6.3.3 Comparison Table

Classical ($\mathbb{R}/\mathbb{Z}$ **or** $\mathbb{Z}/N\mathbb{Z}$)	**Boolean** ($\mathbb{Z}_2^n$)	**Role**
$e^{2\pi ikx}$	$\chi_S(x) = \prod_{i \in S} x_i$	Basis function
Complex-valued	± 1-valued	Basis alphabet
DFT / FFT	WHT / FWHT	Fast algorithm
$O(N \log N)$ complex ops	$O(N \log N)$ integer add/sub	Complexity
$\sum_k \lvert\hat{f}(k)\rvert^2 = \lVert f \rVert_2^2$	$\sum_S \widehat{f}(S)^2 = \lVert f \rVert_2^2$	Parseval
$\widehat{f * g}(k) = \hat{f}(k)\hat{g}(k)$	$\widehat{f * g}(S) = \widehat{f}(S)\widehat{g}(S)$	Convolution
Frequency k	Degree $\lvert S \rvert$	"Frequency"
Low-pass filter	Degree truncation	Smoothing

The Boolean case is simpler in every respect: the basis is real (actually ± 1-valued), the transform uses only additions and subtractions, and the group structure is particularly transparent ($\mathbb{Z}_2^n$ is a product of copies of the simplest nontrivial group). This simplicity is precisely what makes it attractive for binary neural networks.

Connecting the three perspectives. All three topics in this chapter illuminate the same central phenomenon: the Walsh–Fourier basis is the *natural* tool for analyzing binary-valued functions. LTFs show that the most basic computational unit (a threshold gate) has low-degree spectral concentration. Reed–Muller codes show that degree-bounded Walsh expansions form a classical error-correcting code. And the group-theoretic viewpoint explains *why* the Walsh basis is canonical: it consists of exactly the characters of the underlying group $\mathbb{Z}_2^n$, the unique orthonormal basis that simultaneously diagonalizes all translation operators.

Recommended Reading

- **O'Donnell [2014, Chapter 5].** Full proof of Chow's theorem and a detailed treatment of LTF spectral properties (influence bounds, noise sensitivity, the Gotsman–Linial conjecture). Graduate level.
- **MacWilliams and Sloane [1977], *The Theory of Error-Correcting Codes.*** The encyclopedic reference for coding theory. Chapter 13 covers Reed–Muller codes in detail, including decoding algorithms and weight distributions. Graduate level; assumes abstract algebra.

- **Terras [1999], *Fourier Analysis on Finite Groups and Applications.*** Develops Fourier analysis on arbitrary finite groups, with $\mathbb{Z}_2^n$ as a running example. Chapters 1–4 provide the group-theoretic foundation for the character theory in Section 6.3.2. Assumes undergraduate algebra (groups, homomorphisms).

- **Diaconis [1988], *Group Representations in Probability and Statistics.*** A beautifully written monograph connecting group representations to random walks, card shuffling, and statistical applications. Lighter on Boolean analysis but excellent for building intuition about why Fourier analysis on groups works. Assumes basic group theory.

Chapter 7

Application to Binary Neural Networks

This chapter connects the mathematical theory developed in Chapters 1–6 to the specific setting of the thesis: training neural networks with binary (± 1) weights. We summarize the relevant neural network background, explain how the Boolean Fourier framework applies, and trace the expository chain for each key result in the thesis.

7.1 Background: Binary Neural Networks

7.1.1 Standard Neural Networks

A standard feedforward neural network computes a composition of linear transformations and nonlinear activations. For a single layer:

$$y = \sigma(Wx + b),$$

where $W \in \mathbb{R}^{m \times n}$ is a weight matrix, $b \in \mathbb{R}^m$ is a bias vector, and σ is a nonlinear activation function (ReLU, sigmoid, etc.). The parameters W and b are typically stored in 16-bit or 32-bit floating-point format and optimized via gradient descent.

7.1.2 The Transformer Architecture

The Transformer [Vaswani et al., 2017] is the dominant architecture for language modeling. Its layers involve multiple linear projections (query, key, value, output, feedforward), each of the form $Y = WX + b$. For our purposes, the important point is that a Transformer has many weight matrices, and each can be binarized independently.

7.1.3 Binary Weight Networks

A *binary neural network* constrains each weight to ± 1: $W \in \{-1, +1\}^{m \times n}$. This has dramatic implications:

- **Storage:** Each weight requires 1 bit instead of 16 or 32 bits (16× to 32× compression).
- **Computation:** The matrix-vector product Wx for $W \in \{-1,+1\}^{m\times n}$ and $x \in \{-1,+1\}^n$ reduces to additions and subtractions (or XNOR + popcount on binary hardware; see Rastegari et al., 2016 for the implementation details).
- **Training:** The weight space $\{-1,+1\}^{m\times n}$ is discrete, so standard gradient descent does not apply directly.

7.1.4 The Straight-Through Estimator

The standard approach to training binary networks, introduced by Bengio et al. [2013] and applied to binary networks by Courbariaux et al. [2015], uses the *straight-through estimator* (STE):

1. Maintain *latent weights* $W_{\text{real}} \in \mathbb{R}^{m\times n}$ (full-precision).
2. *Forward pass:* Use binary weights $W_{\text{bin}} = \text{sgn}(W_{\text{real}})$.
3. *Backward pass:* Compute gradients with respect to W_{bin}, but *pass them through* to W_{real} as if sgn were the identity: $\frac{\partial \mathcal{L}}{\partial W_{\text{real}}} \approx \frac{\partial \mathcal{L}}{\partial W_{\text{bin}}}$.
4. Update W_{real} with gradient descent.

The STE is a heuristic: the sign function has zero derivative almost everywhere, so the "true" gradient is zero. The STE approximates it by pretending the derivative is 1. Despite this, the STE works remarkably well in practice; see Yin et al. [2019] for a theoretical analysis of why, and Hubara et al. [2016] for extensive empirical evidence.

7.1.5 BitNet

BitNet [Wang et al., 2023] and its successor BitNet b1.58 [Ma et al., 2024] scale binary (and ternary) weight training to the Transformer architecture. Key features:

- Full-precision latent weights during training (STE).
- Per-layer learned scaling factor α (floating-point).
- 8-bit activations (or full-precision during training).

These components constitute "precision leakage": the model is not purely binary even though the deployed weights are.

7.2 The Spectral Perspective on Binary Weights

The thesis proposes viewing each row of a binary weight matrix as a Boolean function and analyzing it via the Walsh–Fourier transform.

7.2.1 The Row-Wise Encoding

Consider a weight matrix $W \in \{-1,+1\}^{d\times d}$ (square, for simplicity; the thesis also handles rectangular matrices). Each row $w_i \in \{-1,+1\}^d$ is a vector of d binary values. (This row-wise viewpoint originates in O'Donnell, 2014, §1.3.)

If $d = 2^n$ (a power of 2), we can view each row as a Boolean function $w_i\colon \{-1,+1\}^n \to \{-1,+1\}$ by identifying the column index $j \in \{0,\dots,d-1\}$ with the hypercube point $\mathrm{bin}(j) \in \{-1,+1\}^n$ via its binary representation. (When d is not a power of 2, the thesis pads rows to the next power of 2; the spectral parameterization then operates on the padded dimension.)

Under this identification, the Walsh–Fourier expansion (Theorem 2.4.1) gives:

$$w_i(j) = \sum_{S\subseteq[n]} \widehat{w}_i(S)\,\chi_S(\mathrm{bin}(j)),$$

and the spectral coefficients can be computed via the FWHT (Algorithm 1) in $O(d\log d)$ time.

7.2.2 Spectral Parameterization

Instead of storing and optimizing the d binary weights directly, the thesis proposes storing the spectral coefficients $\{\widehat{w}_i(S) : |S| \le k\}$ for a truncation degree k. This has $\binom{n}{\le k}$ continuous parameters per row.

At degree $k = 3$ with $d = 1024$ ($n = 10$): $\binom{10}{\le 3} = 1+10+45+120 = 176$ parameters per row, versus $d = 1024$ in the weight domain. This is a $6\times$ compression of the optimization landscape.

During training, the spectral coefficients are optimized by gradient descent. To produce binary weights:

1. Reconstruct the continuous weights via inverse FWHT: $z_i(j) = \sum_{|S|\le k} \widehat{w}_i(S)\,\chi_S(\mathrm{bin}(j))$.
2. Apply the sign function: $w_i(j) = \mathrm{sgn}(z_i(j))$.

At any point during training, the current spectral coefficients define a truncated approximation $z_i = f^{\le k}$ to some underlying Boolean function $f = \mathrm{sgn}(z_i)$. The sign-rounding error bound (Proposition 2.8.2) gives $\mathbb{P}[w_i(j) \ne f(j)] \le \sum_{|S|>k} \widehat{f}(S)^2$, where the tail energy depends on f (which is itself evolving during training).

7.2.3 Normalization Without Learned Parameters

A key contribution of the thesis is replacing BitNet's learned scaling factor α with fixed constants, using two complementary principles:

Parseval normalization (parameter level). By Parseval's identity (Theorem 2.5.1), any Boolean function $f\colon \{-1,+1\}^n \to \{-1,+1\}$ satisfies $\sum_S \widehat{f}(S)^2 = 1$. After the sign projection, the result is always a Boolean function with unit energy, but the *pre-sign* magnitudes of $f^{\leq k}$ matter: they determine the "confidence" of the sign and hence the accuracy of the rounding. Projecting the spectral coefficients onto the unit sphere ($\hat{w} \leftarrow \hat{w}/\|\hat{w}\|_2$) controls these magnitudes by ensuring $\sum_{|S|\leq k} \hat{w}(S)^2 = 1$ throughout training.

CLT normalization (activation level). For any binary weight matrix $W \in \{-1,+1\}^{d\times d}$ and uniform binary input $x \in \{-1,+1\}^d$, each output component $y_j = \sum_{i=1}^d w_{ji}x_i$ is a sum of d independent ± 1 random variables. By Berry–Esséen (Theorem 1.4.2), $y_j/\sqrt{d} \approx \mathcal{N}(0,1)$ for large d.

The required normalization factor $1/\sqrt{d}$ is a fixed constant determined at model design time, not a learned parameter. When $n = \log_2 d$ is even, $1/\sqrt{d} = 2^{-n/2}$ is an exact power of two and can be implemented as a bit-shift (e.g., $d = 1024$ gives $1/\sqrt{d} = 1/32 = 2^{-5}$).

For Hadamard-structured layers ($W = \frac{1}{\sqrt{d}} H_n \cdot \mathrm{diag}(s)$), the $1/\sqrt{d}$ factor is additionally an *exact isometry* ($\|y\|_2 = \|x\|_2$), not just a variance-preserving scaling in expectation.

7.3 Tracing the Expository Chain

We now verify that every mathematical concept and result in the thesis has been covered in this guide, with explicit pointers.

7.3.1 Chapter-by-Chapter Map

Thesis §2.1 (Boolean hypercube). → Section 2.1 (definition), Definition 2.1.3 (Boolean/pseudo-Boolean functions), Definition 2.1.4 (inner product).

Thesis §2.2 (Fourier expansion). → Section 2.2 (parity functions), Theorem 2.3.1 (orthonormality), Theorem 2.4.1 (expansion theorem), Definition 2.4.2 (degree).

Thesis §2.3 (Parseval). → Theorem 2.5.1 (Parseval), Remark 2.5.2 (thesis application).

Thesis §2.4 (LMN theorem). → Section 4.3 (LMN theorem), Section 4.3.1 (circuit complexity background).

Thesis §2.5 (Influence, noise sensitivity, hypercontractivity). → Chapter 3 (all of Chapter 3 of this guide): influence (Definition 3.1.1), total influence (Definition 3.1.3), noise operator (Definition 3.2.1), noise stability (Definition 3.3.1), Bonami–Beckner (Theorem 4.1.1), level-k inequality (Theorem 4.2.1).

Thesis §2.6 (Walsh–Hadamard transform). → Chapter 5 (all of Chapter 5 of this guide): Hadamard matrices (Section 5.1), FWHT (Section 5.4), convolution theorem (Section 5.6).

Thesis §2.7 (Sign activation / LTFs). → Section 6.1 (LTFs), Theorem 6.1.3 (Chow's theorem), Theorem 6.1.5 (bounded noise sensitivity).

Thesis §2.8 (Multi-layer composition). → Section 3.3 (noise stability). When layer 1 computes f and layer 2 computes g, the composed function is $g \circ f$. The key identity is that if the inputs to f are ρ-correlated, the outputs $f(x), f(y)$ are $\mathrm{Stab}_\rho[f]$-correlated (in expectation). Thus noise stability chains across layers: the effective correlation after L layers depends on repeatedly applying the stability function $\rho \mapsto \mathrm{Stab}_\rho[f_\ell]$ for each layer ℓ. (For details on multi-layer composition in the Boolean setting, see O'Donnell, 2014, §2.4 and Chapter 11.)

Thesis §2.9 (Group-theoretic perspective). → Section 6.3 (groups, characters, comparison table).

Thesis §2.9 (Reed–Muller codes). → Section 6.2 (definition, parameters, connection).

Thesis §3.1 (Approximation bound). → Proposition 2.8.2 (sign-rounding error bound), proved using Markov (Theorem 1.3.2) and Parseval (Theorem 2.5.1). The geometric enhancement uses the level-k inequality (Theorem 4.2.1).

Thesis §3.2 (Normalization propagation). → Berry–Esséen (Theorem 1.4.2), orthogonality of Hadamard matrices (Theorem 5.2.1), Kronecker product properties (Section 1.2).

Thesis §4 (Architecture). → Section 7.1 (BNN background), Section 7.1.4 (STE), Section 7.1.5 (BitNet), Section 7.2.2 (spectral parameterization), Section 7.2.3 (normalization).

7.4 Worked Example: Spectral Parameterization of One Row

We trace the full pipeline of the thesis's spectral parameterization for a single row of a binary weight matrix, using concrete numbers.[1]

Setup. Let $d = 8$ ($n = 3$). Consider a row $w = (+1, -1, -1, +1, -1, +1, +1, -1) \in \{-1, +1\}^8$. We view w as a Boolean function $f\colon \{-1, +1\}^3 \to \{-1, +1\}$ by identifying column index $j \in \{0, \ldots, 7\}$ with $\mathrm{bin}(j) \in \{-1, +1\}^3$.

Step 1: Compute the Walsh–Fourier coefficients. Apply the FWHT (Algorithm 1) to w, then divide by $N = 8$. The three rounds of butterfly operations produce:

$$\begin{aligned}
&\text{Round 0 } (h{=}1): \quad (0,\ 2,\ 0,\ -2,\ 0,\ -2,\ 0,\ 2),\\
&\text{Round 1 } (h{=}2): \quad (0,\ 0,\ 0,\ 4,\ 0,\ 0,\ 0,\ -4),\\
&\text{Round 2 } (h{=}4): \quad (0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 8).
\end{aligned}$$

Dividing by $N = 8$: all coefficients are 0 except $\widehat{f}(\{1, 2, 3\}) = 1$ (index $7 = 111_2$, corresponding to $S = \{1, 2, 3\}$). So $f = \chi_{\{1,2,3\}}$, the parity function on 3 bits. All energy is at degree 3.

Step 2: Degree-k truncation. At truncation degree $k = 1$: $f^{\leq 1}(x) = 0$ (all coefficients at degrees 0 and 1 are zero). Since $\mathrm{sgn}(0)$ is undefined (or assigned arbitrarily), the truncation gives no useful information. The sign-rounding error bound (Proposition 2.8.2) gives $\mathbb{P}[\mathrm{sgn}(f^{\leq 1}) \neq f] \leq \sum_{|S|>1} \widehat{f}(S)^2 = 1$. The bound is vacuous—as expected, since parity is the worst case for low-degree approximation.

Step 3: Contrast with majority. The Maj_3 function from Example 2.6.3 has 75% of its energy at degree 1 and 25% at degree 3. The degree-≤ 1 truncation captures most of the spectral energy, and $\mathrm{sgn}(f^{\leq 1})$ agrees with f on all 8 inputs (as shown in Example 2.8.3).

Step 4: Spectral reconstruction. In the thesis pipeline, we store only the low-degree coefficients $\{\widehat{w}(S) : |S| \leq k\}$, reconstruct the continuous weights via the inverse FWHT (which uses the *same* matrix H_n), then apply the sign function. For the degree-1 truncation of the Maj_3 example from Example 2.8.3:

1. Store: $\widehat{f}(\emptyset) = 0$, $\widehat{f}(\{1\}) = \widehat{f}(\{2\}) = \widehat{f}(\{3\}) = 1/2$.

[1] See companion notebook, §6: end-to-end pipeline (NumPy and PyTorch), parameter count comparison, and CLT normalization histogram.

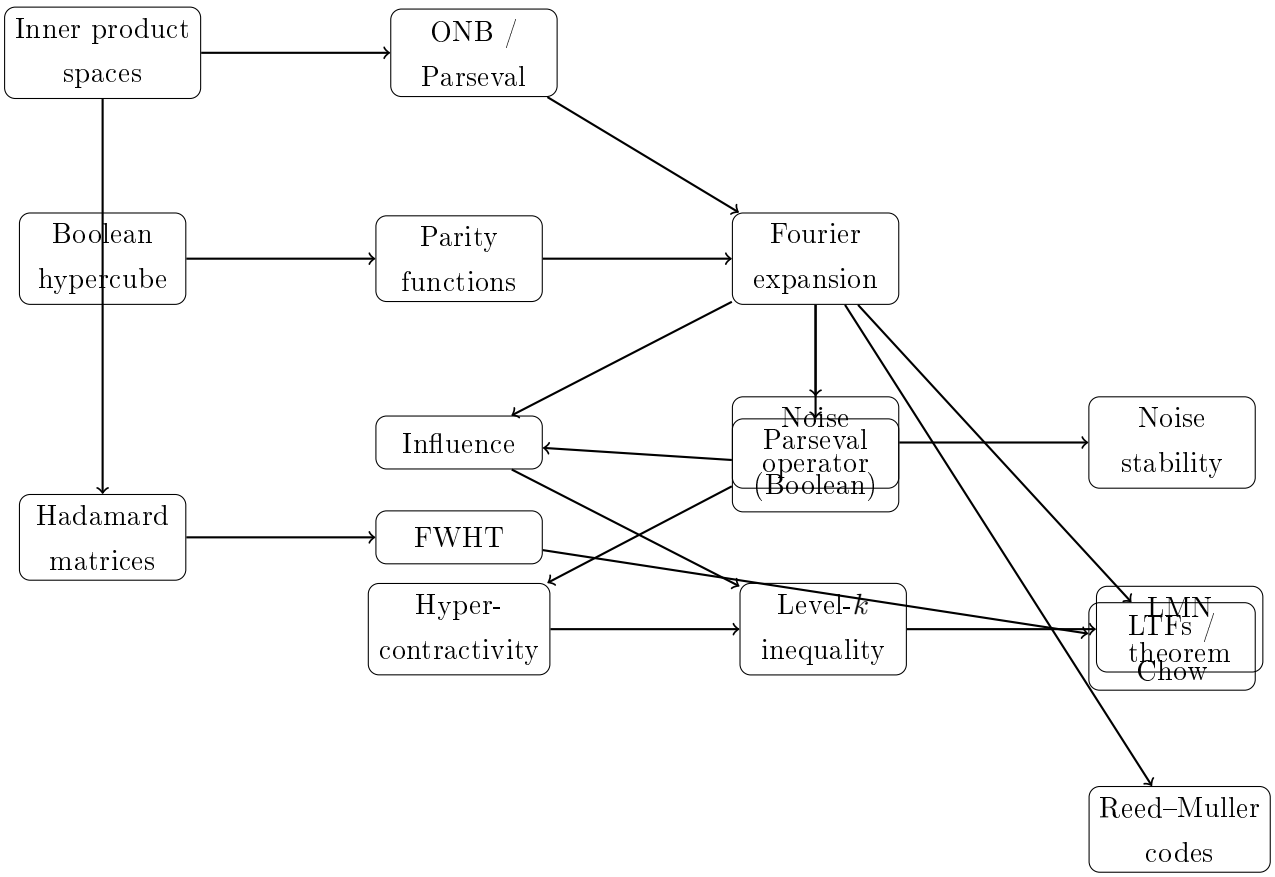
2. Reconstruct: $f^{\leq 1}(x) = \frac{1}{2}(x_1 + x_2 + x_3)$.
3. Sign: $\mathrm{sgn}(f^{\leq 1}(x)) = \mathrm{sgn}(x_1 + x_2 + x_3) = \mathrm{Maj}_3(x)$.

The reconstruction is *exact* in this case (0% error), even though we discarded the degree-3 coefficient $\widehat{f}(\{1,2,3\}) = -1/2$. The spectral parameterization uses 4 parameters (degree ≤ 1) instead of 8 weights, a $2\times$ compression.

Takeaway. Spectrally concentrated functions (like majority) can be faithfully recovered from their low-degree coefficients, while spectrally dispersed functions (like parity) cannot. The thesis hypothesizes that trained binary weights resemble the former, not the latter.

7.5 Concept Dependency Graph

The following shows the logical dependencies among the key concepts. An arrow $A \to B$ means "A is used in the proof/definition of B."



7.6 Suggestions for Further Reading

Suggested reading path. Start with de Wolf [2008] for a quick overview (item 2 below), then O'Donnell [2014] Chapters 1–5 for the core theory (item 1). For the BNN context,

read Courbariaux et al. [2015] and Wang et al. [2023] (item 9). The remaining items can be consulted as needed.

1. **Boolean Fourier analysis (comprehensive):** O'Donnell [2014]. Available free at arXiv:2105.10386. Chapters 1–5 cover the core material; Chapters 8–10 cover hypercontractivity and the LMN theorem.

2. **Boolean Fourier analysis (short introduction):** de Wolf [2008]. A 30-page survey covering the essentials. Freely available at the Theory of Computing library.

3. **Noise sensitivity and social choice:** Benjamini et al. [1999]. The foundational paper connecting noise sensitivity to percolation theory. Beautiful mathematics.

4. **Hadamard matrices:** Horadam [2007]. Comprehensive treatment of Hadamard matrices and their applications.

5. **Walsh functions and signal processing:** Beauchamp [1984]. The classical reference for Walsh functions in engineering.

6. **Reed–Muller codes:** MacWilliams and Sloane [1977], Chapters 13–14. The standard reference for algebraic coding theory.

7. **Fourier analysis on finite groups:** Terras [1999]. Develops Fourier analysis on general finite groups with many applications.

8. **Probability (CLT, Berry–Esséen):** Durrett [2019], Chapter 3. Or Feller [1971], Volume 2, Chapter XVI.

9. **Binary neural networks:** Start with Courbariaux et al. [2015], then Rastegari et al. [2016] and Hubara et al. [2016] for the foundational work. Wang et al. [2023] and Ma et al. [2024] for the state of the art.

Bibliography

Miklós Ajtai. Σ_1^1-formulae on finite structures. *Annals of Pure and Applied Logic*, 24(1): 1–48, 1983. doi: 10.1016/0168-0072(83)90038-6.

Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

Sheldon Axler. *Linear Algebra Done Right*. Springer, 4th edition, 2024. doi: 10.1007/978-3-031-41026-0.

Kenneth G. Beauchamp. *Applications of Walsh and Related Functions*. Academic Press, 1984. ISBN 0-12-084180-2.

William Beckner. Inequalities in Fourier analysis. *Annals of Mathematics*, 102(1):159–182, 1975. doi: 10.2307/1970980.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013.

Itai Benjamini, Gil Kalai, and Oded Schramm. Noise sensitivity of Boolean functions and applications to percolation. *Publications Mathématiques de l'IHÉS*, 90:5–43, 1999. doi: 10.1007/BF02698830. arXiv: math/9811157.

Andrew C. Berry. The accuracy of the Gaussian approximation to the sum of independent variates. *Transactions of the American Mathematical Society*, 49(1):122–136, 1941. doi: 10.2307/1990053.

Aline Bonami. Étude des coefficients de Fourier des fonctions de $L^p(G)$. *Annales de l'Institut Fourier (Grenoble)*, 20(2):335–402, 1970.

C. K. Chow. On the characterization of threshold functions. In *Switching Circuit Theory and Logical Design, Proceedings of the Second Annual Symposium*, pages 34–38. IEEE, 1961. doi: 10.1109/FOCS.1961.26.

James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.

Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. BinaryConnect: Training deep neural networks with binary weights during propagations. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pages 3123–3131, 2015.

Tri Dao, Beidi Chen, Nimit S. Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 4690–4721. PMLR, 2022.

Ronald de Wolf. A brief introduction to Fourier analysis on the Boolean cube. *Theory of Computing Library, Graduate Surveys*, (1), 2008. doi: 10.4086/toc.gs.2008.001. Freely available at `https://theoryofcomputing.org/articles/gs001/`.

Persi Diaconis. *Group Representations in Probability and Statistics*, volume 11 of *Lecture Notes—Monograph Series*. Institute of Mathematical Statistics, 1988.

Rick Durrett. *Probability: Theory and Examples*. Cambridge University Press, 5th edition, 2019. doi: 10.1017/9781108591034.

Carl-Gustav Esséen. On the Liapounoff limit of error in the theory of probability. *Arkiv för Matematik, Astronomi och Fysik*, 28A(9):1–19, 1942.

William Feller. *An Introduction to Probability Theory and Its Applications*, volume 2. John Wiley & Sons, 2nd edition, 1971.

Merrick Furst, James B. Saxe, and Michael Sipser. Parity, circuits, and the polynomial-time hierarchy. In *Mathematical Systems Theory*, volume 17, pages 13–27, 1984. doi: 10.1007/BF01744431. Conference version in FOCS 1981.

Leonard Gross. Logarithmic Sobolev inequalities. *American Journal of Mathematics*, 97 (4):1061–1083, 1975. doi: 10.2307/2373688.

Jacques Hadamard. Résolution d'une question relative aux déterminants. *Bulletin des Sciences Mathématiques*, 17:240–246, 1893.

Prahladh Harsha, Adam R. Klivans, and Raghu Meka. Bounding the sensitivity of polynomial threshold functions. *Theory of Computing*, 10:1–26, 2014. doi: 10.4086/toc.2014.v010a001.

Kathy J. Horadam. *Hadamard Matrices and Their Applications*. Princeton University Press, 2007. ISBN 978-0-691-11921-2.

Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2nd edition, 2012. ISBN 978-0-521-54823-6.

Johan Håstad. *Computational Limitations of Small-Depth Circuits.* PhD thesis, Massachusetts Institute of Technology, 1987. Also available as: MIT Press, 1987.

Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *Advances in Neural Information Processing Systems 29 (NeurIPS)*, pages 4107–4115, 2016.

Jeff Kahn, Gil Kalai, and Nathan Linial. The influence of variables on Boolean functions (extended abstract). In *Proceedings of the 29th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 68–80. IEEE Computer Society, 1988. doi: 10.1109/SFCS.1988.21923.

Eyal Kushilevitz and Yishay Mansour. Learning decision trees using the Fourier spectrum (extended abstract). In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 455–464. ACM, 1991. doi: 10.1145/103418.103466.

Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, Fourier transform and learnability. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 574–579. IEEE Computer Society, 1989. doi: 10.1109/SFCS.1989.63527.

Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, Fourier transform, and learnability. *Journal of the ACM*, 40(3):607–620, 1993. doi: 10.1145/174130.174138. Journal version of the FOCS 1989 paper.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint*, 2024.

Florence Jessie MacWilliams and Neil James Alexander Sloane. *The Theory of Error-Correcting Codes.* North-Holland, 1977. ISBN 0-444-85193-3.

Yishay Mansour. Learning Boolean functions via the Fourier transform. In *Theoretical Advances in Neural Computation and Learning*, volume 275 of *The Springer International Series in Engineering and Computer Science*, pages 391–424. Springer, 1994. doi: 10.1007/978-1-4615-2686-6_15.

Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry.* MIT Press, 1969. Expanded edition, 1988.

David E. Muller. Application of Boolean algebra to switching circuit design and to error detection. *IRE Transactions on Electronic Computers*, EC-3(3):6–12, 1954. doi: 10.1109/IRELEC.1954.6499441.

Ryan O'Donnell. Analysis of Boolean functions (lecture notes). Carnegie Mellon University, course 15-859S, 2007. Available at `https://www.cs.cmu.edu/~odonnell/boolean-analysis/`.

Ryan O'Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014. ISBN 978-1-107-03832-5. doi: 10.1017/CBO9781139814782. The standard graduate-level textbook. Free edition: arXiv:2105.10386.

Yuval Peres. Noise stability of weighted majority. *arXiv preprint*, 2004.

Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *Computer Vision – ECCV 2016*, volume 9908 of *Lecture Notes in Computer Science*, pages 525–542. Springer, 2016.

Alexander A. Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical Notes of the Academy of Sciences of the USSR*, 41(4):333–338, 1987. doi: 10.1007/BF01137685.

Irving S. Reed. A class of multiple-error-correcting codes and the decoding scheme. *IRE Transactions on Information Theory*, 4(4):38–49, 1954. doi: 10.1109/TIT.1954.1057465.

John L. Shanks. Computation of the fast Walsh–Fourier transform. *IEEE Transactions on Computers*, C-18(5):457–459, 1969. doi: 10.1109/T-C.1969.222685.

Irina G. Shevtsova. On the absolute constants in the Berry–Esséen type inequalities. *Doklady Mathematics*, 84(3):862–864, 2011. doi: 10.1134/S1064562411060328.

Audrey Terras. *Fourier Analysis on Finite Groups and Applications*. Cambridge University Press, 1999. ISBN 978-0-521-45718-7. doi: 10.1017/CBO9780511626265.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 5998–6008, 2017.

Joseph L. Walsh. A closed set of normal orthogonal functions. *American Journal of Mathematics*, 45(1):5–24, 1923. doi: 10.2307/2387224.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint*, 2023.

Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley J. Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. In *7th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2019.

Index

AND function, 17

Berry–Esséen theorem, 11
binary neural network, 50
BitNet, 51
Bonami–Beckner inequality, 28
Boolean function, 13
butterfly operation, 38

central limit theorem, 10
character (group), 47
Chebyshev's inequality, 10
Chow parameters, 43
Chow's theorem, 43
convolution
    dyadic, 21

degree (Fourier), 16
dictator function, 17

fast Walsh–Hadamard transform (FWHT), 38
Fourier coefficient, 16
Fourier expansion, 16

Hadamard matrix, 35
hypercontractivity, 28
hypercube, 13

influence, 22
inner product, 8

Kronecker product, 9

level-k inequality, 29
linear threshold function, 42
LMN theorem, 32

majority function, 18
Markov's inequality, 10

noise operator, 23
noise sensitivity, 25
noise stability, 25

orthonormality, 15

parity function, 14
Parseval's identity, 16
pseudo-Boolean function, 13

Reed–Muller code, 45

sign-rounding error, 20
spectral parameterization, 52
spectral weight, 29
straight-through estimator, 51

total influence, 23

Walsh–Hadamard transform, 37

About This Guide

Boolean Fourier Analysis and the Walsh–Hadamard Transform provides a self-contained, proof-complete exposition of the mathematics behind spectral methods for binary neural networks. Starting from undergraduate linear algebra and probability, it develops the Fourier analysis of Boolean functions on the hypercube $\{-1, +1\}^n$, covers influence theory, noise sensitivity, and hypercontractivity, presents the fast Walsh–Hadamard transform, and explores connections to Reed–Muller codes, linear threshold functions, and the representation theory of $\mathbb{Z}_2^n$. The final chapter traces every concept back to its application in training neural networks with binary weights.

Features:

- Full proofs and worked examples for every major result.
- A companion Jupyter notebook with NumPy and PyTorch implementations of all key algorithms.
- Chapter-by-chapter map linking the mathematical theory to the thesis on spectral methods for binary neural networks.
- Concept dependency graph and curated reading lists.

Written as a companion to the thesis Spectral Methods for Binary Neural Networks, *this guide serves both as a reference for researchers at the intersection of Boolean analysis and machine learning, and as a self-study introduction for students encountering these ideas for the first time.*